# DATA MINING

## PRACTICAL 1

**Date Given** : **10$^{TH}$ MARCH 2026**
**Submission Date** : **3$^{RD}$ APRIL 2026**
**Date Submitted** : **8$^{TH}$ APRIL 2026**
**Weightage** : **20%**
**Semester** : **FEBRUARY 2026**
**Lecturer** : **AHMAD LUQMAN BIN AHMAD KAMAL ARIFFIN**

**Instruction to students:**

- This is a Group Assignment.
- Complete this cover sheet and attach it to your Assignment (first page).

**Student declaration:**

***We declare that:***

- ***This assignment is our own work***
- ***We understand what is meant by plagiarism***
- ***Our lecturer has the right to deduct our marks in the case of:***
  - ***Late submission***

***Any plagiarism found in our assignment.***

| ***MUHAMMAD HARISH HAQIM BIN ADNAN*** | ***AFNAN FARISH BIN ABDUL RAHIM*** | ***KUVALESAN A/L RAVICHANDRAN*** | ***SURIANA BINTI ABDULLAH*** |
|---|---|---|---|
| ***012025020434*** | ***012025021256*** | ***012025021382*** | ***012025021263*** |
|  |  |  |  |

| **MARKS:** | **Total** | |
|---|---|---|
| | | **100** |

# ACKNOWLEDGEMENT

We the members of Group A, would like to express our heartfelt gratitude to our lecturer, Mr Ahmad Luqman Bin Ahmad Kamal Ariffin, for his invaluable guidance and encouragement throughout the course. The lecturer's explanation and support during the lectures have significantly improved the group's comprehension of the various data cleaning and transformation techniques. The lecturer's feedback has also been instrumental in shaping the quality of the work produced by the group members.

We also wish to express our gratitude to the entire group members, Afnan Farish, Harish Haqim, Kuvalesan, and Suriana, for their collaboration and efforts in the production of the assignment for Data Mining (CCS21703) subject. The entire group members were instrumental in the production of the assignment. The spirit of cooperation and the entire group members' efforts ensured that the group was able to successfully complete the practical.

We would also wish to express our gratitude to our peers and families for their encouragement during the production of the report. The encouragement from the peers and families motivated the entire group members to successfully complete the assignment.

# TABLE OF CONTENTS

1.0 INTRODUCTION TO DATA MINING ........ 1
1.1 Definition ........ 1
1.2 Significance of Data Mining ........ 1
1.3 Three Major Tasks of Data Mining ........ 2
1.3.1 Classification ........ 2
1.3.2 Clustering ........ 3
1.3.3 Association Rule Mining ........ 4
1.4 Real-World Application of Data Mining ........ 6
1.4.1 Healthcare ........ 6
1.4.2 Retail ........ 7
1.4.3 Education ........ 8
2.0 DATA TYPES AND DATA UNDERSTANDING ........ 10
2.1 Introduction ........ 10
2.2 Difference Between Different Data Types ........ 10
2.2.1 Categorial Data ........ 10
2.2.2 Numerical Data ........ 10
2.2.3 Ordinal Data ........ 11
2.3 The Importance of Understanding Different Data Types ........ 11
2.4 Two Types of Visualizations ........ 13
2.4.1 Histogram for Data Distributions ........ 13
2.4.2 Scatter Plot for Data Relationships ........ 14
3.0 DATA PREPROCESSING ........ 17
3.1 Introduction ........ 17
3.2 Overview of The Dataset ........ 17
3.3 Three Common Issues from the Dataset ........ 19
3.3.1 Missing Values ........ 20
3.3.2 Data Inconsistencies ........ 21
3.3.3 Outliers Data ........ 22

3.4 The Importance of Data Processing for Success of Data Mining Tasks ................. 23
3.5 Methods for Handling Dataset Issues ................. 24
3.5.1 Missing Values ................. 24
3.5.2 Inconsistencies ................. 26
3.5.3 Noise and Outliers ................. 27
3.5.4 Normalisation of Numerical Attributes ................. 35
3.6 Improvements of Accuracy and Reliability of Data Mining Models ................. 38
3.6.1 Handling Missing Values ................. 38
3.6.2 Correcting Data Inconsistencies ................. 38
3.6.3 Removing Outliers ................. 38
3.6.4 Normalization of Numerical Attributes ................. 38
3.6.5 Ensuring Fairness and Reproducibility ................. 38
3.6.6 Overall Impact ................. 39
3.7 Insights from Data Analysis ................. 39
3.7.1 Basic Statistical Analysis ................. 39
4.0 PRESENTATION AND COMMUNICATION ................. 45
5.0 CONCLUSION ................. 50
6.0 REFERENCES ................. 51

# LIST OF FIGURES

Figure 1.1: Types of classification ................. 2
Figure 1.2: Example of cluster technique ................. 3
Figure 1.3: How association rules works ................. 4
Figure 1.4: Example of clustered data to detect outbreaks for infectious organism ................. 6
Figure 1.5: Categorical data distribution in retail ................. 7
Figure 1.6: Three rules for each category associated with its accuracy ................. 8
Figure 2.1: The histograms for COVID-19 daily cases, daily deaths, and daily recovered in Egypt and Saudi Arabia ................. 13
Figure 2.2: The estimated pdf for Saudi Arabia COVID-19 daily cases data ................. 14
Figure 2.3: The plot of number of confirmed cases of COVID-19 and healthcare capacities (Number of hospital beds or medical doctors) across the Africa continent ................. 15

Figure 3.1: Coding of the overview dataset with output ........................................................17
Figure 3.2: Basic information about the dataset..........................................................................18
Figure 3.3: The dataset information...............................................................................................18
Figure 3.4: The dataset datatypes ...................................................................................................19
Figure 3.5: Total null values from each column ..........................................................................20
Figure 3.6: Rows with at least one null value ...............................................................................20
Figure 3.7: Gender column with three inconsistency values .......................................................21
Figure 3.8: StudyProgram column with an inconsistency value and null value.......................21
Figure 3.9: FinalScore column with data inconsistencies ............................................................21
Figure 3.10: ParentEducation column with data inconsistencies ................................................22
Figure 3.11: Age column with outliers data..................................................................................22
Figure 3.12: StudyHoursPerWeek column with outliers data .....................................................22
Figure 3.13: Absences column with outliers data with missing value .......................................23
Figure 3.14: Program code and output for copying data frame .................................................24
Figure 3.15: Filling missing values column with mode or median ...........................................25
Figure 3.16: Fixing the columns with inconsistencies .................................................................26
Figure 3.17: Program code and output for cleaning the noise and outliers..............................27
Figure 3.18: Displaying dataset and its data types of each columns ..........................................28
Figure 3.19: Displaying the values of verified cleaning columns ...............................................29
Figure 3.20: Displaying the value counts from cleaning columns (Part 1) ................................30
Figure 3.21: Displaying the value counts from cleaning columns (Part 2) ................................31
Figure 3.22: Displaying the value counts from cleaning columns – FinalScore .......................32
Figure 3.23: Displaying the value counts from cleaning columns - MidtermScore (Part 1) ...33
Figure 3.24: Displaying the value counts from cleaning columns - MidtermScore (Part 2) ...33
Figure 3.25: Displaying the value counts from cleaning columns - MidtermScore (Part 3) ...34
Figure 3.26: Displaying the value counts from cleaning columns - MidtermScore (Part 4) ...34
Figure 3.27: Normalising the five columns ...................................................................................35
Figure 3.28: Creating and saving new dataset files and display cleaned dataset..................36
Figure 3.29: Displaying the normalised dataset (Part 1)..............................................................37
Figure 3.30: Displaying the normalised dataset (Part 2)..............................................................37
Figure 3.31: Basic statistical analysis.............................................................................................39
Figure 3.32: Program code for creating pie chart and bar chart ................................................40
Figure 3.33: Percentages of Student by Study Program (left) and Total Students by Gender (right) .................................................................................................................................................40
Figure 3.34: Program code for creating multiple scatter plots ...................................................41
Figure 3.35: Study Hours vs Absences (left) and Study Hours vs Final Score (right)............42
Figure 3.36: Program code for creating scatter plot and clustered bar graphs .......................43

Figure 3.37: Study Hours vs Mid Term Score (left) and Comparison of Parent Education Levels by Study Program (right)........................................................................................44

# EXTERNAL LINKS FOR PROGRAM CODES, DOCUMENTS AND DATASETS

Google Drive Link: https://drive.google.com/drive/folders/1FhGH57CFXXWMHyTWwU7hV23B2pJ60IWS?usp=sharing

GitHub Link: https://github.com/HaqimSaintis/DataMining-Practical-1.git

# EXTERNAL LINKS FOR SLIDES PRESENTATION

Canva Link: https://canva.link/46nvecaxyurjkh8

Google Drive Link: https://drive.google.com/drive/folders/1FhGH57CFXXWMHyTWwU7hV23B2pJ60IWS?usp=sharing

# 1.0 INTRODUCTION TO DATA MINING

## 1.1 Definition

Data mining can thus be defined as the application of machine learning and statistical analysis in extracting significant patterns, relationships, and valuable information from large volumes of data (Holdsworth, n.d.). It is also referred to as knowledge discovery in databases because it enables organizations to derive valuable information from unprocessed data.

The development of machine learning, data warehousing, and big data technologies has greatly contributed to the growth of data mining in recent decades. This is because it has enabled organizations to mine large volumes of complex data, although there are still issues of efficiency in the technology (Holdsworth, n.d.).

Generally, there are two major uses of data mining technology: describing and predicting. Describing involves analysing the data in order to summarize its significant features, while predicting involves applying algorithms in order to forecast the trends of the data in the future (Holdsworth, n.d.). For instance, descriptive analysis might reveal information regarding customer purchasing habits, while prediction might estimate the sales made in the near future.

The integration of artificial intelligence and machine learning has further added to the capabilities of data mining. These technologies have been found to automate much of the analysis process, and organizations have been able to efficiently detect fraud, monitor users, and detect performance issues and potential security breaches. Technologies like Apache Spark and other data analytics platforms have been found to simplify the process by allowing organizations to efficiently process and visualize data, making it easier for decision-makers to access information (Holdsworth, n.d.).

In summary, data mining has been found to have evolved to become an essential part of modern data analysis. By integrating statistical analysis, machine learning, and artificial intelligence, organizations have been able to efficiently transform raw data into knowledge that is useful to the organization (Holdsworth, n.d.).

## 1.2 Significance of Data Mining

Data mining is important because it allows organizations to turn raw data into valuable intelligence. This is possible because data mining can help businesses find hidden patterns and relationships in the data they generate. This ability to turn raw data into valuable intelligence is what makes data mining important and a core part of a contemporary analytics strategy (Ramakrishnan, 2023).

Besides problem-solving, data mining is important in the context of historical and real-time analytics. This is because the technique can help organizations implement business intelligence tools that can analyse historical data and real-time analytics tools that can analyse the data being created at a given time. This ability to analyse historical and real-time data can help organizations respond to changes in the market and the behaviour of the company's customers and the challenges facing the company's operations.

Another important advantage of using the data mining technique is the ability to improve the efficiency of organizational operations. This is the focus of the study: the impact of the technique on various aspects of organizational operations and functions such as marketing, sales, supply chain management, finance, and customer support.

Lastly, the importance of data mining can be seen in its applicability to different industries. For instance, in healthcare, data mining can be used for diagnosis and research; in detail, data mining can be used for recommendations and marketing; in manufacturing, data mining can be used for fraud detection and risk management, among other uses. Therefore, data mining, through its ability to give insights for improved strategies and decision-making, can give organizations a competitive edge, more returns, and sustainability (Ramakrishnan, 2023).

## 1.3 Three Major Tasks of Data Mining

### *1.3.1 Classification*

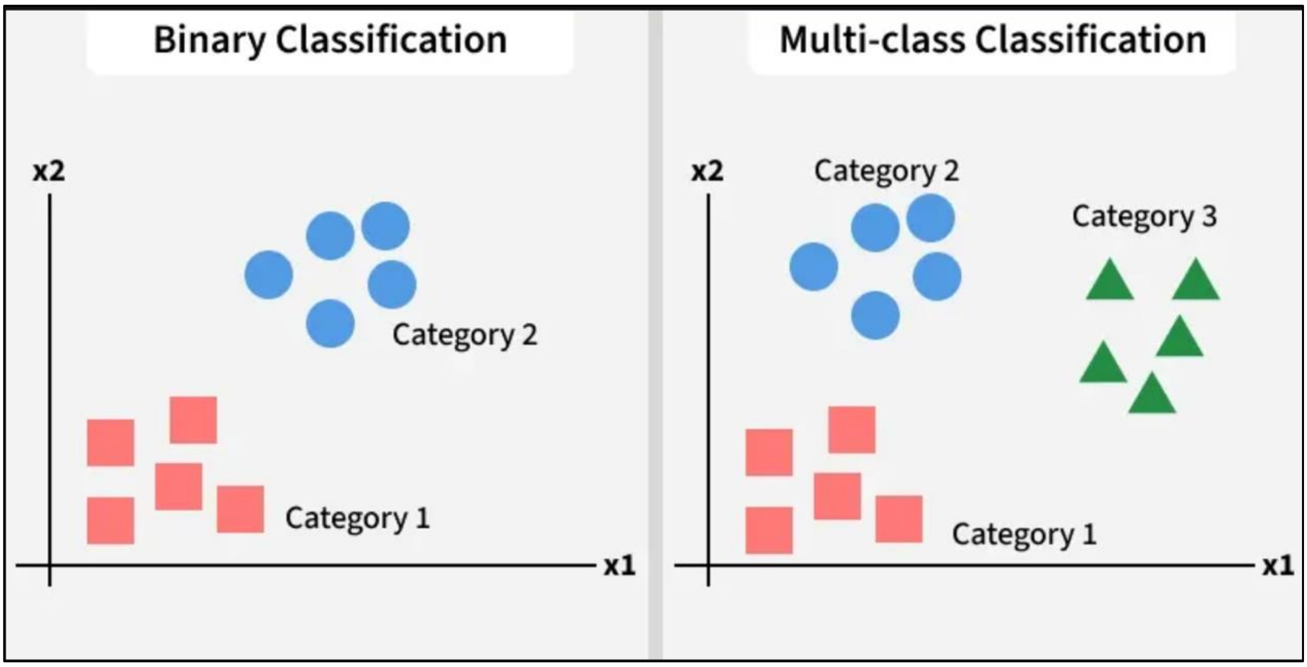



**Figure 1.1:** Types of classification

In Figure 1.1 made by GeeksForGeeks (2026), the classes of objects are pre-defined according to the needs of an organization, including definitions of the characteristics that objects have in common. This allows for easier analysis of the underlying data (Holdsworth, n.d.).

The classification in the context of data mining is defined as the process of categorization of the data based on the predefined categories. There are two different types of classification. In the case of binary classification, the data is categorized based on the two different categories. For instance, the classification process in the context of customer buying behavior would involve the categorization of the customers based on the fact that they will buy the products or will not buy the products.

On the other hand, in the context of multi-class classification, the categorization of the customers would be based on the fact that the customers will buy a particular type of products, such as shoes, shirts, or accessories. These two different types of classification processes in the context of data mining are considered to be the most important because they enable the organizations to analyze the complex data and take decisions based on the same (Holdsworth, n.d.).

For example, a consumer products company can review its couponing campaign by analysing historical coupon redemption data, sales data, inventory statistics, and consumer data they have collected to determine the best approach for a future couponing campaign.

### *1.3.2 Clustering*

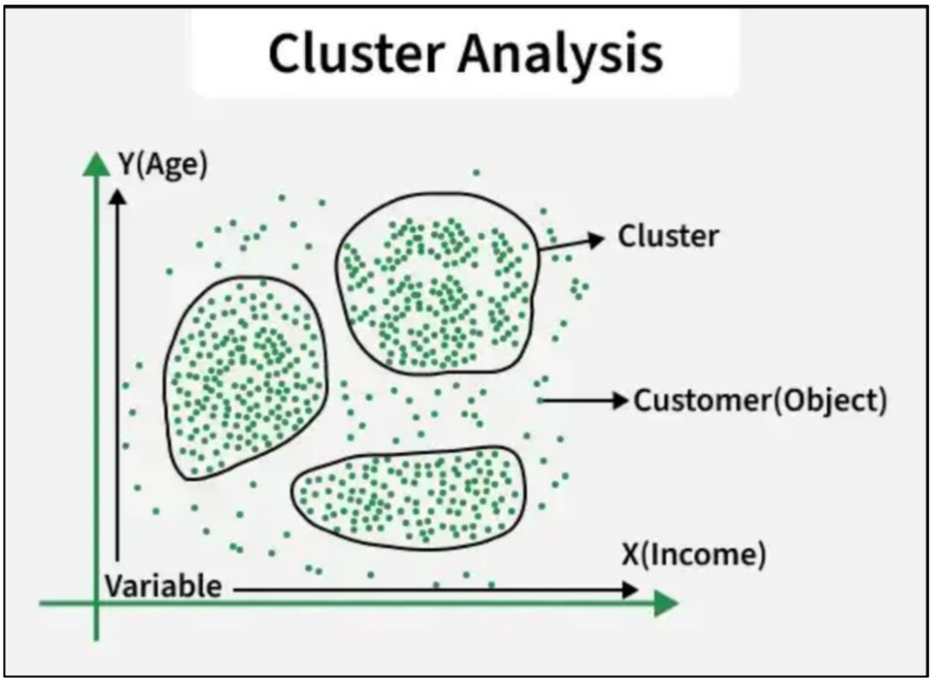



**Figure 1.2:** Example of cluster technique

As shown in Figure 1.2 made by GeeksForGeeks (2026), similar to classification, clustering also shows similar groups, and then goes on to show more groups based on differences. For a soap manufacturer, the classification groups might be preset such as detergent, bleach, laundry softener, floor cleaner, and floor wax, while clustering might result in groups such as laundry and floor care (Holdsworth, n.d.).

The term clustering in data mining is defined as the collection of similar data objects without considering previously defined classes. Unlike classification, clustering does not require previously defined classes. Clustering is used for exploratory purposes and helps in discovering hidden patterns in the data. For instance, a soap company might have previously classified its products into different classes like detergent soap and bleach soap. Clustering might help in discovering classes like laundry soap and floor soap.

Clustering can also help in segmenting customers according to different criteria like age and income, as shown in the diagram above. Clustering is important because it helps in discovering hidden classes in the data, which might not have been previously discovered by the organization (Holdsworth, n.d.).

### *1.3.3 Association Rule Mining*


**Figure 1.3:** How association rules works

Based on Figure 1.3 made by Hashemi-Pour et al. (2024), association rule is an if-then, rule-based approach to discover the relationships between the variables in the data set. The strength of the relationships is determined based on the truthfulness of the if-then statements. The support level is determined based on the frequency of the appearance of the related variables (Holdsworth, n.d.).

Association rule mining is a technique used in the field of data mining for discovering relationships between items in large databases. It operates on the basis of an if-then rule, in which the occurrence of an item leads to the occurrence of another. This is measured in terms of various parameters, including the measure of support, which refers to the rate at which two items appear together in the database.

For instance, in a database of six transactions, cereal appears in three of the transactions (support 50%), milk appears in four of the transactions (support 67%), and cereal and milk appear together in three of the transactions (support 50%). This indicates that customers purchasing cereal have a high likelihood of purchasing milk as well.

Association rule mining helps organizations discover hidden relationships in the purchasing habits of consumers, which in turn helps them develop better marketing strategies and improve sales performance (Holdsworth, n.d.).

These approached are commonly used in market basket analysis, which allows companies to gain better insight into the relationships between the different products, including those products commonly purchased together.

## 1.4 Real-World Application of Data Mining

### *1.4.1 Healthcare*

Characteristics of Outbreaks[a]

| No. | Date | Organism | Cluster: No. of Related Isolates | Molecular Typing Method | Duration of Transmission, Days | Transmission Route | Infections Potentially Prevented: 7-Day Intervention | Infections Potentially Prevented: 14-Day Intervention |
|---|---|---|---|---|---|---|---|---|
| 1 | Feb 12 | *Acinetobacter baumannii* | 3 | PFGE | 19 | Trauma ICU | 1 | 1 |
| 2 | Mar 13 | *Klebsiella pneumoniae* | A: 28 | PFGE | 865 | ERCP | 20 | 20 |
| | | | B: 2 | | 3 | ERCP | 0[b] | 0[b] |
| | | | C: 2 | | 13 | ERCP | 0[b] | 0[b] |
| | | | Total: 32 | | | | | |
| 3 | Jun 15 | *K. pneumoniae* | 7 | PFGE | 29 | Bronchoscope | 5 | 3 |
| 4 | Jul 15 | *Pseudomonas aeruginosa* | 8 | PFGE | 42 | Bronchoscope | 5 | 4 |
| 5 | Aug 15 | *A. baumannii* | 5 | PFGE | 80 | Medical ICU | 3 | 2 |
| 6 | Dec 15 | *P. putida* | 3 | PFGE | 1 | Bronchoscope | 0 | 0 |
| 7 | Apr 16 | *K. pneumoniae* | 9 | PFGE & WGS | 39 | Gastroscope | 5 | 3 |
| 8 | Jun 16 | *Clostridium difficile* | A: 2 | WGS | 4 | Trauma Floor | 0[b] | 0[b] |
| | | | B: 2 | | 15 | Post Anesthesia Unit | 0[b] | 0[b] |
| | | | C: 2 | | 35 | Pulmonology Floor | 0[b] | 0[b] |
| | | | Total: 6 | | | | | |
| 9 | Sep 16 | *C. difficile* | 4 | WGS | 67 | Medical ICU | 1 | 1 |
| | | | | | | | Total: 40 | Total: 34 |

**Figure 1.4:** Example of clustered data to detect outbreaks for infectious organism

As shown in Figure 1.4 made by Sundermann et al. (2019), data mining is an essential tool for healthcare organizations because it helps them convert medical data into useful information. It is commonly used for disease diagnosis and prediction, where medical data is mined to identify patterns and predict disease progression. It is also used for treatment optimization, where data mining helps healthcare organizations learn from patient outcomes and provide more effective treatment (Kolling, Furstenau, Sott, Rabaioli, Ulmi, Bragazzi, & Tedesco, 2021).

Data mining is also used for patient management in healthcare organizations. For instance, data mining can be applied to patient records to monitor patients more effectively. In this regard, data mining can help reduce medical errors. In terms of its application at a larger scale, data mining can be used for public health and epidemiology. It can be applied for disease outbreaks and can help in policy development (Kolling et al., 2021).

Operational efficiency is another area where data mining can be applied in healthcare organizations. Data mining can be applied for scheduling and for reducing costs in healthcare organizations. Data mining can be applied for detecting fraud in billing and insurance claims. It can be applied for risk management (Kolling et al., 2021).

In general, data mining in the healthcare industry plays an important role in the diagnosis, treatment, and performance of the healthcare system. By identifying hidden patterns and relationships in medical data, data mining helps generate valuable insights to enhance the quality and efficiency of the healthcare system (Kolling et al., 2021).

### *1.4.2 Retail*

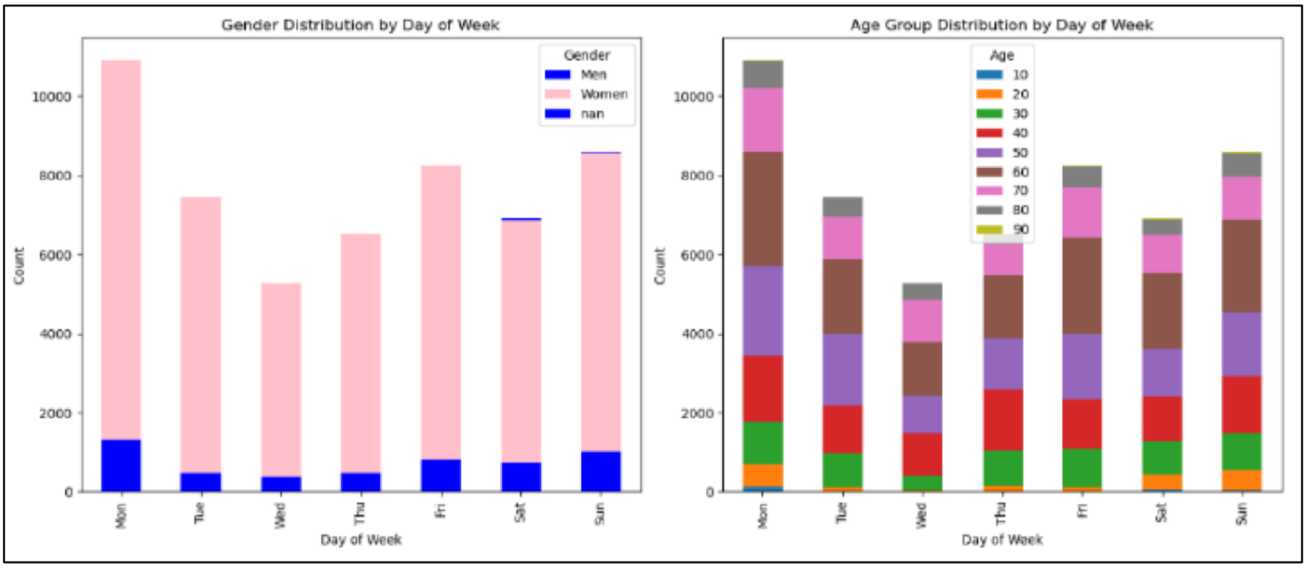



**Figure 1.5:** Categorical data distribution in retail

As demonstrated in Figure 1.5 made by Kholod et al. (2024), data mining is an important tool for the retail industry, as it helps organizations convert vast amounts of customer transactional data into useful knowledge. The retail industry is a competitive industry, and data mining is an important tool for retailers to make better and more effective campaigns for the market and to attract profitable customers. It also helps retailers to predict the requirements of the customers and offer discounts to retain the existing customers while attracting new ones.

The most important application of data mining for the retail industry is the analysis of the market basket, where the application helps retailers to know the products that are often purchased together. It is also an important tool for retailers to improve the convenience for the customers and to enhance sales. Another important application is customer segmentation and target marketing, where the application uses clustering to segment the customers based on similar behaviour.

The study also highlights various applications of data mining for the retail industry, such as Walmart, Procter & Gamble, Macy's, J Crew, and Neiman Marcus, and how these applications are important for improving the marketing strategy, customer loyalty, and gaining a competitive edge for the retail industry.

In general terms, the importance of data mining in retail businesses is significant, considering that it helps businesses to identify profitable customers, maximize campaigns, and sustain success in the marketplace. Therefore, businesses are able to make informed decisions using techniques such as reward-based targeting, segmentation, and predictive modeling to achieve long-term success (Ramageri & Desai, 2013).

### *1.4.3 Education*

| Category | Rule | Accuracy |
|---|---|---|
| **Excellent** | 1. Final = Good -> Mid-term = Excellent -> CT-2 = Excellent -> Individual Presentation = Excellent | 95.82% |
| | 2. Final = Excellent -> Mid-term = Excellent | 91.37% |
| | 3. Final = Excellent -> Mid-term = Good -> Individual Presentation = Excellent-> CT-2 = Excellent | 89.13% |
| **Good** | 1. Final = Good -> Mid-term = Good | 90.64% |
| | 2. Final = Good -> Mid-term = Medium: Good -> Group Assignment = Excellent \|\| Good | 88.13% |
| | 3. Final = Medium -> Mid-term = Good -> Individual Presentation = Excellent | 91.51% |
| **Medium** | 1. Final = Medium -> Mid-term = Bad | 93.65% |
| | 2. Final = Bad -> Mid-term = Medium | 92.76% |
| | 3. Final = Bad -> Individual Presentation = Excellent ->Mid-term = Medium \|\| Bad | 88.17% |
| **Bad** | 1. Final = Bad -> Individual Presentation = Bad -> Mid-term = Bad | 98.45% |
| | 2. Final = Bad -> Individual Presentation = Medium -> Mid-term = Bad | 85.71% |
| | 3. Final = Bad -> Individual Presentation = Good -> Mid-term = Bad -> CT-2 = Bad -> CT-3 = Bad | 87.5 % |

**Figure 1.6:** Three rules for each category associated with its accuracy

Figure 1.6 from Ashaduzzman et al. (2018) for instance, data mining has been recognized as an effective means in the education sector, specifically through the subdiscipline of EDM. One of its most impactful applications is in student performance prediction, where data mining techniques can be utilized to predict grades even before enrolment in courses or examinations. By using various data mining techniques such as classification, clustering, regression, and neural networks, educators can predict student performance.

The study highlights that data mining can not only be utilized for predicting grades but can also be employed for early intervention. By using data mining techniques, teachers can identify students who may not perform well at an early stage and can provide them with additional help to increase their chances of success. Moreover, data mining can be utilized for personalized learning, where various approaches can be taken to cater to individual students based on their predicted performance.

Apart from specific student outcomes, data mining also plays an important role in the improvement of the curriculum. Data mining, using vast amounts of student performance data, enables schools and educational institutions to improve the curriculum. For instance, clustering algorithms enable the categorization of students with similar learning styles. This enables educators to develop more differentiated approaches to student learning. Regression analysis also enables educators to identify the factors that have the greatest impact on student performance.

In conclusion, the use of data mining in the educational sector is beneficial to both the student and the educator. Data mining offers insights that are beneficial to student learning and the improvement of the curriculum. Data mining enables the creation of more efficient learning systems (Zhang et al., 2021).

# 2.0 DATA TYPES AND DATA UNDERSTANDING

## 2.1 Introduction

This paper will discuss the difference between categorical, numerical and ordinal data types to understand the difference between three different data types. And, it will also discuss the importance of understanding data types before performing the data mining tasks. Finally, it will provide two suggestions of visualization types so that we can understand the data distributions and relationships alongside with their purpose.

## 2.2 Difference Between Different Data Types

### *2.2.1 Categorial Data*

According to Rumsey (2021), Categorical data represent characteristics or groups rather than numbers. It identifies the items by labels, names and categories. Categorial data organize the information into distinct, non-numerical groups such as a person's gender, marital status, hometown, or the types of movies they like. Categorical data can also take on numerical values such as "1" indicating male and "2" indicating female, but those numbers don't have mathematical meaning. We couldn't add them together. Other names for categorical data are qualitative data and nominal data. This type of data has no inherent ordering. The measure of central tendency is only Mode and no median and no mean.

There are three characteristics of categorial data which are there is no inherent order where the categories are just labels with no implied hierarchy or ranking, it is the type of qualitative data where it describe the attributes and characteristics rather than numerical values and finally there is an equality check where the main operation for nominal data is checking whether the two categories are the same or different (Habib, 2024).

### *2.2.2 Numerical Data*

Numerical data or also known as quantitative data represents the measurable quantities. This data can take on numeric values that allow for arithmetic operations like addition, subtraction, or averaging. Numerical data can be divided into two main types such as continuous data and discrete data (Rumsey, 2021).

1. Continuous Data

A continuous data is a numerical data that can take any value within a given range. It is measured, not counted, and can take on an infinite number of possible values, including fractions and decimals. There are three characteristics of continuous data. Firstly, it has infinite possible values which can take any value within a specified range. Secondly, it is measurable rather than counted. Finally, the data is meaningful where the difference between the values are known and consistent. Example of continuous data is the height measurement of a person which is 5.72 feet, 6.03 feet and more (Habib, 2024).

2. Discrete Data

A discrete variable is a numerical variable that represents countable values. It has a finite or countable number of possible values, typically whole numbers. This type of data can take on possible values that can be listed out where the list of possible values may be fixed, or it can go to infinity. There are three characteristics of discrete data. Firstly, it has finite possible values where it can only take specific and countable values. Secondly, it is countable where the data values are the result of counting rather than measuring. Finally, it is only whole numbers meaning there is no fractions, rational or irrational numbers. An example of discrete data is total students inside the mathematics class everyday (Habib, 2024).

### *2.2.3 Ordinal Data*

Ordinal data mixes numerical and categorical data. The data fall into categories, but the numbers placed on the categories have meaning such as order and ranking. Ordinal data are often treated as categorical, where the groups are ordered when graphs and charts are made. However, unlike categorical data, the numbers do have mathematical meaning. The categories can be arranged in a sequence, but the exact differences between them are not measurable. We can compare ranks, but we cannot quantify the exact gap between categories. (Rumsey, 2021).

There three characteristics of ordinal data. Firstly, it is inherent order where there is a clear ranking and order among the categories. Secondly, there is qualitative within the ranking of the data in other words there is implied hierarchy. Finally, there is rank comparison where we can compare the categories in terms of “greater” or “lesser”, but the intervals between the data are not known or consistent. An example of ordinal data is customer satisfaction where there is the level of “Poor”, “Fair”, “Good”, “Very Good” and “Excellent” (Habib, 2024).

## 2.3 The Importance of Understanding Different Data Types

Understanding the difference between data types is crucial before performing the data mining tasks to provide the valuable and cleaned dataset in the future because they directly determine which analytical techniques are valid, ensure accurate preprocessing, and prevent erroneous, misleading results. There are few reasons to justify this importance (Voroshchuk, 2021).

1. Ensuring Data Quality and Accuracy

Misinterpreting a data type such as treating a numerical code as a continuous measurement will leads to faulty, biased models. Understanding types ensures data is cleaned and handled appropriately such as handling missing numerical values with imputation, which is not suitable for text data. It can also help in the process of cleaning and validating data. Correct data types ensure data integrity and prevent Garbage In Garbage Out (GIGO). For instance, if we know that the column “Price” should be numeric data, we can easily identify and clean the entries such as “ten” or “hundred” that should be numeric (Wicaksono, 2025).

2. Defining Valid Operations

All different data types have their own corresponding mathematical and logical operation that can be perform where not all data types can do arithmetic operations and not all data types can used logical operations. If we misidentify the data type inside a data type such as treating the postal code of customers as a numeric number instead of text will produce errors and irrelevant results when we do mathematical calculations (Wicaksono, 2025).

3. Statistical Analysis Calculation

Statistical analysis affects how we calculate among various data types which will affect on how we calculate the statistics and probabilities from the data to be put inside the insights and data visualizations charts and plots. Selecting the wrong statistical calculation for a particular data type can end up in invalid calculations and errors. For example, for numeric data, we can calculate the mean, median, standard deviation, and perform regression tests. For nominal categorical data, we can only calculate the frequency or mode. Finally, for ordinal categorical data, we can calculate the median, but it may not be appropriate to calculate the mean (Wicaksono, 2025).

4. Choosing The Correct Data Visualizations

Choosing the right chart depends on the data type. The type of chart or graph that is most effective for presenting data is largely determined by its data type. Nominal data works well with bar charts and pie charts, while continuous, interval data is best represented with histograms or scatter plots (Wicaksono, 2025).

5. Improving Computational Efficiency

At a technical level, databases and analytics software allocate storage space and optimize query performance based on data types. Storing and processing data in the correct format saves memory and speeds up computations. For example, treating column “Price” as text instead of integers makes calculations slow and memory-intensive (Wicaksono, 2025).

## 2.4 Two Types of Visualizations

There are two types of visualizations that can be used to understand data distributions and relationships with their purpose. Selecting the right data type can impact query speed and storage efficiency, which is important when working with big data (IBM, n.d.).

### *2.4.1 Histogram for Data Distributions*

A histogram is a chart that plots the distribution of numeric variable's values as a series of bars. Each bar typically covers a range of numeric values called a bin or class; a bar's height indicates the frequency of data points with a value within the corresponding bin. Histograms are good for showing general distributional features of dataset variables. We can see roughly where the peaks of the distribution are, whether the distribution is skewed or symmetric, and if there are any outliers. It helps identify the shape of the data such as normal or bell-shaped or skewed, its center, spread or variability, and the presence of outliers or multiple modes (peaks) (Cunha, 2024).

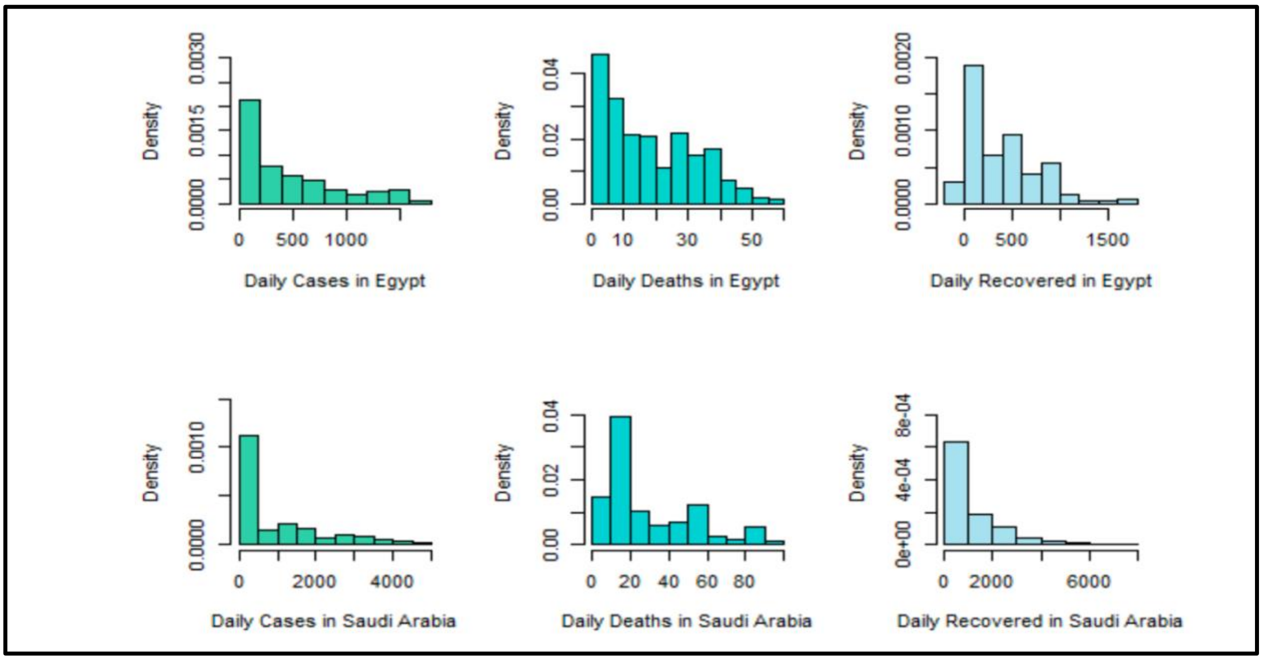



**Figure 2.1:** The histograms for COVID-19 daily cases, daily deaths, and daily recovered in Egypt and Saudi Arabia

Figure 2.1 shows the example of histogram taken from Mansour et al. (2021), it shows the histograms for COVID-19 daily cases, daily deaths, and daily recovered in Egypt and Saudi Arabia. The y-axis shows the density of having cases or deaths while the x-axis shows the total number of cases and deaths within the range.

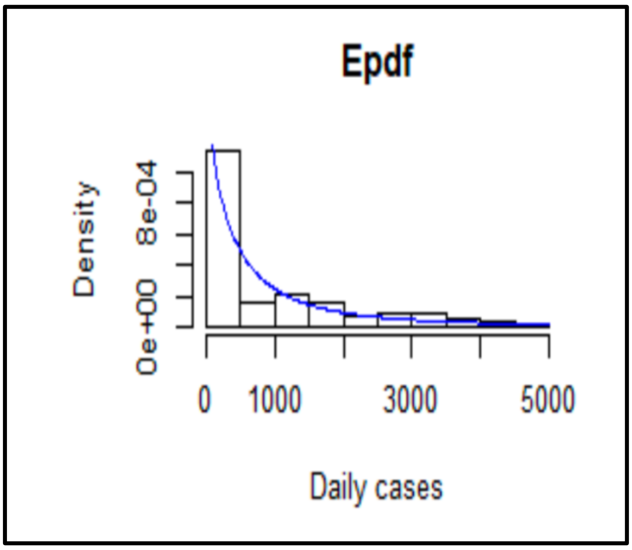



**Figure 2.2:** The estimated pdf for Saudi Arabia COVID-19 daily cases data

Figure 2.2 taken from Mansour et al., (2021) shows the estimated pdf for Saudi Arabia COVID-19 daily cases data. The histogram shows the density of COVID-19 cases in Saudi Arabia. The skew line shows its going from high to low as the x-axis expands, meaning the density is going to be lower as the number of daily cases grow.

### *2.4.2 Scatter Plot for Data Relationships*

Scatter plots display data as a collection of points on a graph to show the relationship between two variables, with one variable determining the point's position on the horizontal axis (x) and the other on the vertical axis (y). Additional variables can be represented by adjusting the color, shape or size of each point. Scatter plots help identify trends or clusters within a data set, making it easier to spot correlations or outliers (Cunha, 2024).

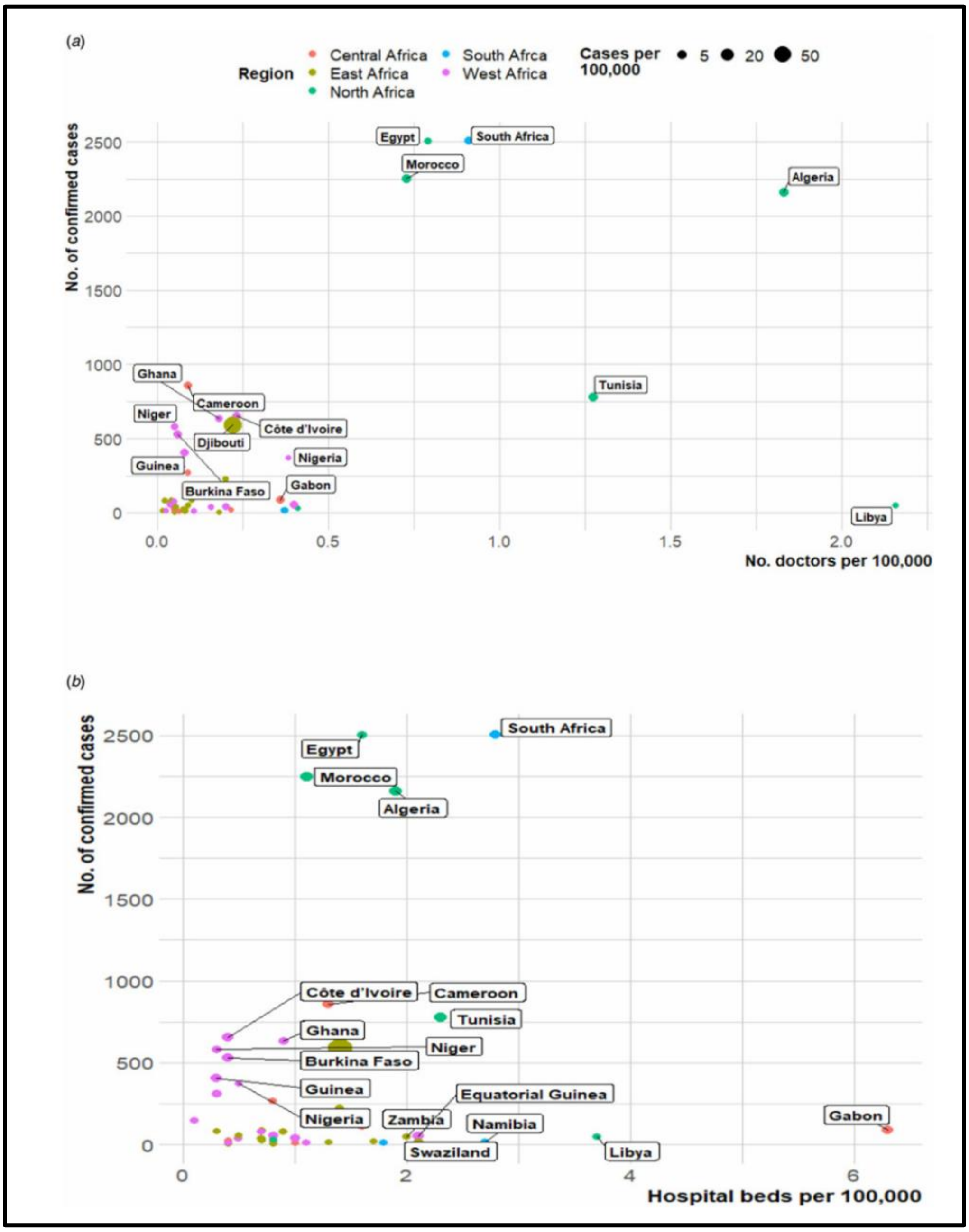



**Figure 2.3:** The plot of number of confirmed cases of COVID-19 and healthcare capacities (Number of hospital beds or medical doctors) across the Africa continent

Figure 2.3 shows the example of scatter plot taken from Gayawan et al. (2020). It is about the plot of number of confirmed cases of COVID-19 and healthcare capacities (Number of hospital beds or medical doctors) across the Africa continent. The y-axis shows the number of confirmed cases while the x-axis shows the number of hospital beds per 100,000. In Figure 2(a), Libya has the highest number of doctors while having the low number of confirmed cases. On the other hand, South Africa has high number of confirmed cases which is more than 2500 and has the number of doctors between 0.5 and 1.0. In Figure 2(b), South Africa plot tells us that it has high number of confirmed cases with over hundreds of thousand beds between 2 and 4. Gabon has the highest number of hospital beds with lower number of confirmed cases.

# 3.0 DATA PREPROCESSING

## 3.1 Introduction

Data preprocessing the process of cleaning and normalizing the datasets before showing the insights to the stakeholders. This is a very crucial part in data mining.

## 3.2 Overview of The Dataset

Part (a)

```
[1]: # import pandas library
     import pandas as pd
     pd.__version__
```

```
[1]: '2.3.3'
```

```
[2]: # Disable future warning
     pd.set_option('future.no_silent_downcasting', True)
     # Read and load raw data
     df = pd.read_excel('Edu_data.xlsx')
     df
```

[2]:

| | StudentID | Gender | Age | StudyProgram | StudyHoursPerWeek | Absences | MidtermScore | FinalScore | ParentEducation | FinalGrade |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | STU001 | Female | 20 | Computer Science | 12 | 2.0 | 78 | 85 | Bachelor | Pass |
| 1 | STU002 | Male | 19 | Business | 8 | 5.0 | 65 | 70 | High School | Pass |
| 2 | STU003 | Female | 21 | Engineering | 15 | 0.0 | 88 | 92 | Master | Pass |
| 3 | STU004 | M | 150 | Arts | 5 | 12.0 | 45 | 50 | Bachelor | Fail |
| 4 | STU005 | Female | 22 | Computer Science | 20 | 1.0 | 95 | 98 | PhD | Pass |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 195 | STU196 | Male | 20 | Arts | 5 | 9.0 | 53 | 58 | High School | Fail |
| 196 | STU197 | Female | 21 | Computer Science | 22 | 0.0 | 97 | 99 | PhD | Pass |
| 197 | STU198 | Male | 19 | Business | 11 | 4.0 | 74 | 78 | Bachelor | Pass |
| 198 | STU199 | Female | 20 | Engineering | 14 | 3.0 | 82 | 85 | Master | Pass |
| 199 | STU200 | Male | 21 | Arts | 6 | 5.0 | 56 | 61 | High School | Fail |

200 rows × 10 columns

**Figure 3.1:** Coding of the overview dataset with output

Figure 3.1 shows the code and output for overview of the dataset. Firstly, we import the pandas library and verify the library's version. After that, we disable any future down casting warning from the pandas library. Then, variable df use pandas to read the raw data and display the raw data inside the cell.

Get basic information

```
[3]: # Print size of the dataset
print(f'The dataset has {df.shape[0]} rows and {df.shape[1]} columns')
```

```
The dataset has 200 rows and 10 columns
```

```
[4]: # Print dataset head
df.head()
```

[4]:

| | StudentID | Gender | Age | StudyProgram | StudyHoursPerWeek | Absences | MidtermScore | FinalScore | ParentEducation | FinalGrade |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | STU001 | Female | 20 | Computer Science | 12 | 2.0 | 78 | 85 | Bachelor | Pass |
| 1 | STU002 | Male | 19 | Business | 8 | 5.0 | 65 | 70 | High School | Pass |
| 2 | STU003 | Female | 21 | Engineering | 15 | 0.0 | 88 | 92 | Master | Pass |
| 3 | STU004 | M | 150 | Arts | 5 | 12.0 | 45 | 50 | Bachelor | Fail |
| 4 | STU005 | Female | 22 | Computer Science | 20 | 1.0 | 95 | 98 | PhD | Pass |

**Figure 3.2:** Basic information about the dataset

Figure 3.2 shows the basic information about the dataset where we use the shape function to display the confirmed total rows and total columns of the dataset. The total rows are 200 and the total columns are 10. Next, we use the head function to display the top five rows of the dataset.

```
[5]: # Get dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   StudentID          200 non-null    object
 1   Gender             200 non-null    object
 2   Age                200 non-null    int64
 3   StudyProgram       195 non-null    object
 4   StudyHoursPerWeek  200 non-null    int64
 5   Absences           199 non-null    float64
 6   MidtermScore       200 non-null    int64
 7   FinalScore         194 non-null    object
 8   ParentEducation    200 non-null    object
 9   FinalGrade         200 non-null    object
dtypes: float64(1), int64(3), object(6)
memory usage: 15.8+ KB
```

**Figure 3.3:** The dataset information

Figure 3.3 shows the dataset information where it displays the information such as dataframe's class, columns, non-null counts, total rows and columns, data types and the memory usage.

```
[6]: # Get dataset datatypes
     print(f'Dataset datatypes:\n\n{df.dtypes}')

     Dataset datatypes:

     StudentID             object
     Gender                object
     Age                    int64
     StudyProgram          object
     StudyHoursPerWeek      int64
     Absences             float64
     MidtermScore           int64
     FinalScore            object
     ParentEducation       object
     FinalGrade            object
     dtype: object
```

**Figure 3.4:** The dataset datatypes

Figure 3.4 shows the dataset datatypes where there over three different datatypes such as object, int64, and float64.

### 3.3 Three Common Issues from the Dataset

There are over three common issues that were found inside the dataset by using Python programming language with Jupyter Notebook from Anaconda Navigator Software.

### 3.3.1 *Missing Values*

Get information about null values

```
[7]: # Display total null values in each column
print(f'Total null values from each column:\n\n{df.isnull().sum()}')
```

```
Total null values from each column:

StudentID           0
Gender              0
Age                 0
StudyProgram        5
StudyHoursPerWeek   0
Absences            1
MidtermScore        0
FinalScore          6
ParentEducation     0
FinalGrade          0
dtype: int64
```

```
[8]: # Count total rows with at least 1 null value
print(f'Total rows with at least 1 null value is {df.isnull().any(axis=1).sum()}')
```

```
Total rows with at least 1 null value is 11
```

**Figure 3.5:** Total null values from each column

Figure 3.5 shows the total null values from each column where StudyProgram column contains 5 null values, Absences column contains only 1 null value, and finally the FinalScore column contain 6 null values. The sum of total null values that were found from the dataset is 12 while the total rows that have at least 1 null value is 11 only.

```
[9]: # Display rows with null values
df[df.isnull().any(axis=1)]
```

[9]:

| | StudentID | Gender | Age | StudyProgram | StudyHoursPerWeek | Absences | MidtermScore | FinalScore | ParentEducation | FinalGrade |
|---|---|---|---|---|---|---|---|---|---|---|
| **7** | STU008 | Male | 21 | Arts | 6 | 8.0 | 55 | NaN | Master | Fail |
| **9** | STU010 | Male | 19 | NaN | 9 | 1.0 | 68 | 74 | Bachelor | Pass |
| **19** | STU020 | Male | 21 | Arts | 6 | 5.0 | 999 | NaN | High School | Pass |
| **39** | STU040 | Male | 19 | Arts | 6 | NaN | 59 | NaN | Bachelor | Pass |
| **44** | STU045 | Female | 20 | NaN | 21 | 1.0 | 93 | 96 | PhD | Pass |
| **78** | STU079 | Female | 21 | NaN | 14 | 2.0 | 82 | 85 | Master | Pass |
| **79** | STU080 | Male | 22 | Arts | 6 | 6.0 | 58 | NaN | High School | Pass |
| **118** | STU119 | Female | 20 | NaN | 14 | 2.0 | 81 | 84 | Master | Pass |
| **119** | STU120 | Male | 21 | Arts | 6 | 6.0 | 60 | NaN | High School | Pass |
| **158** | STU159 | Female | 20 | NaN | 14 | 2.0 | 83 | 86 | Master | Pass |
| **159** | STU160 | Male | 21 | Arts | 6 | 6.0 | 59 | NaN | High School | Pass |

**Figure 3.6:** Rows with at least one null value

Figure 3.6 list the rows that contain at least one null value. By using the function ".isnull()" and ".axis(=1)" it search the rows that contains at least one null value from the dataset.

### *3.3.2 Data Inconsistencies*

```
[11]: # Gender column
      df['Gender'].unique()

[11]: array(['Female', 'Male', 'M', 'F', 'm'], dtype=object)
```

**Figure 3.7:** Gender column with three inconsistency values

Figure 3.7 shows the Gender column that has data inconsistencies. By using the function ".unique()", it will displays the unique values from the column Gender. The value "Female" is the same as "F" and the value "Male" is the same as value "M" and "m".

```
[13]: # StudyProgram column
      df['StudyProgram'].unique()

[13]: array(['Computer Science', 'Business', 'Engineering', 'Arts', nan, -10],
            dtype=object)
```

**Figure 3.8:** StudyProgram column with an inconsistency value and null value

Figure 3.8 shows the StudyProgram column that has data inconsistencies and null value. By using the function ".unique()", it will displays the unique values from the column StudyProgram. The value -10 is the error value and nan value is the null value

```
[17]: # FinalScore column
      df['FinalScore'].unique()

[17]: array([85, 70, 92, 50, 98, 72, nan, 78, 74, 95, 45, 84, 65, 89, 55, 99,
             76, 88, 86, 71, 87, 48, 97, 75, 90, 60, 80, 69, 94, 52, 73, 57, 83,
             93, 49, 96, 58, 77, 46, 66, 56, 62, 51, 91, 59, 81, 53, 79, 47, 67,
             100, 64, 61, 82, 68, 54, 63, 'Eighty'], dtype=object)
```

**Figure 3.9:** FinalScore column with data inconsistencies

Figure 3.9 shows the FinalScore column that has data inconsistencies. By using the function ".unique()", it will displays the unique values from the column FinalScore. The value "80" is the same as "Eighty".

```
[18]: # ParentEducation column
      df['ParentEducation'].unique()

[18]: array(['Bachelor', 'High School', 'Master', 'PhD', 'Bachelors', 'BSc'],
            dtype=object)
```

**Figure 3.10:** ParentEducation column with data inconsistencies

Figure 3.10 shows the ParentEducation column that has data inconsistencies. By using the function ".unique()", it will displays the unique values from the column ParentEducation. The value "Bachelor" is the same as "Bachelors" and the value "BSc" is the short term for "Bachelor Science" where it does not give full name.

### *3.3.3 Outliers Data*

```
[12]: # Age column
      df['Age'].unique()

[12]: array([ 20,  19,  21, 150,  22,  -2])
```

**Figure 3.11:** Age column with outliers data

Figure 3.11 shows the Age column that has outliers data. By using the function ".unique()", it will displays the unique values from the column Age. There is the age over 150 years old and the age with -2 years old.

```
[14]: # StudyHoursPerWeek column
      df['StudyHoursPerWeek'].unique()

[14]: array([ 12,   8,  15,   5,  20,  10,  14,   6,  11,   9,  18,   4,  13,
               7,  16, 999,  19,  17,  22,  21])
```

**Figure 3.12:** StudyHoursPerWeek column with outliers data

Figure 3.12 shows the StudyHoursPerWeek column that has outliers data. By using the function ".unique()", it will displays the unique values from the column StudyHoursPerWeek. There is the hours which is 999 hours very high and not normal compare to others.

```
[15]: # Absences column
      df['Absences'].unique()

[15]: array([ 2.,  5.,  0., 12.,  1.,  3.,  8.,  4., 15.,  6., -5., 10., 45.,
              7., 11.,  9., nan, 13., 14.])
```

**Figure 3.13:** Absences column with outliers data with missing value

Figure 3.13 shows the Absences column that has outliers data. By using the function ".unique()", it will displays the unique values from the column Absences. There is the absences -5 value and missing value.

## 3.4 The Importance of Data Processing for Success of Data Mining Tasks

Data preprocessing is a step in data mining. The quality of the data directly affects the results. In real-world situations raw data is often imperfect. It may have missing values, inconsistent formats, duplicate records, noise or outliers. If these issues are not fixed before analysis data mining may produce patterns, weak predictions or misleading conclusions. Therefore preprocessing helps prepare the dataset for analysis and model building (Han et al., 2012)

Data preprocessing makes the dataset more consistent and structured. When values are in the format software can read and analyse the data more effectively. For example if "Male" "male" and "M" represent the category the system may treat them as different values. This reduces analysis quality. Correcting issues during preprocessing makes the dataset cleaner more organized and easier to interpret (Witten et al., 2016).

Preprocessing is important because it increases the reliability and efficiency of data mining tasks. Clean data reduces errors during analysis. Allows algorithms to focus on meaningful relationships in the dataset. This improves project success because findings become more accurate, valid and useful, for decision-making. In short data preprocessing is necessary to ensure data mining tasks are successful and produce results (Han et al., 2012).

## 3.5 Methods for Handling Dataset Issues

### *3.5.1 Missing Values*

Part (c)

Missing Values

```
[20]: # Made a copy dataset so that it won't modify the original dataset
df_cleaning = df.copy()
df_cleaning
```

[20]:

| | StudentID | Gender | Age | StudyProgram | StudyHoursPerWeek | Absences | MidtermScore | FinalScore | ParentEducation | FinalGrade |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | STU001 | Female | 20 | Computer Science | 12 | 2.0 | 78 | 85 | Bachelor | Pass |
| **1** | STU002 | Male | 19 | Business | 8 | 5.0 | 65 | 70 | High School | Pass |
| **2** | STU003 | Female | 21 | Engineering | 15 | 0.0 | 88 | 92 | Master | Pass |
| **3** | STU004 | M | 150 | Arts | 5 | 12.0 | 45 | 50 | Bachelor | Fail |
| **4** | STU005 | Female | 22 | Computer Science | 20 | 1.0 | 95 | 98 | PhD | Pass |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **195** | STU196 | Male | 20 | Arts | 5 | 9.0 | 53 | 58 | High School | Fail |
| **196** | STU197 | Female | 21 | Computer Science | 22 | 0.0 | 97 | 99 | PhD | Pass |
| **197** | STU198 | Male | 19 | Business | 11 | 4.0 | 74 | 78 | Bachelor | Pass |
| **198** | STU199 | Female | 20 | Engineering | 14 | 3.0 | 82 | 85 | Master | Pass |
| **199** | STU200 | Male | 21 | Arts | 6 | 5.0 | 56 | 61 | High School | Fail |

200 rows × 10 columns

**Figure 3.14:** Program code and output for copying data frame

Figure 3.14 shows the program code and output for copying data frame. In order to avoid change the original raw dataset, we apply ".copy()" function to the df_cleaning variable to create a copy from the dataset dataframe to apply data cleaning later.

```
[21]: # Fill missing values column with mode or median

      # StudyProgram (categorical - use mode)
      df_cleaning['StudyProgram'] = df_cleaning['StudyProgram'].fillna(df_cleaning['StudyProgram'].mode()[0])

      # Absences (numerical - use median)
      df_cleaning['Absences'] = df_cleaning['Absences'].fillna(df_cleaning['Absences'].median())

      # FinalScore (categorical - use mode))
      df_cleaning['FinalScore'] = df_cleaning['FinalScore'].fillna(df_cleaning['FinalScore'].mode()[0])

      print("Total missing values after cleaning:\n")
      print(df_cleaning.isnull().sum())

      Total missing values after cleaning:

      StudentID          0
      Gender             0
      Age                0
      StudyProgram       0
      StudyHoursPerWeek  0
      Absences           0
      MidtermScore       0
      FinalScore         0
      ParentEducation    0
      FinalGrade         0
      dtype: int64
```

**Figure 3.15:** Filling missing values column with mode or median

Figure 3.15 shows the program code and output for filling missing values with mode or median. We will use the function ".fillna" and ".mode()" follow by index 0 to mark the mode Firstly, the StudyProgram column is the categorical data so it will use mode value to fill the missing values. Secondly, the Absences column is a numerical data so it will use median value to fill the missing values. Next, the FinalScore is a categorial data so it will use mode value to fill the missing values.

### 3.5.2 *Inconsistencies*

Inconsistencies

```
[22]: # Fix Gender
df_cleaning['Gender'] = df_cleaning['Gender'].replace({
    'M': 'Male',
    'm': 'Male',
    'F': 'Female'
})

# Fix ParentEducation
df_cleaning['ParentEducation'] = df_cleaning['ParentEducation'].replace({
    'Bachelors': 'Bachelor',
    'BSc': 'Bachelor'
})

# Fix StudyProgram invalid value
df_cleaning['StudyProgram'] = df_cleaning['StudyProgram'].replace(-10, df_cleaning['StudyProgram'].mode()[0])

# Fix FinalScore text
df_cleaning['FinalScore'] = df_cleaning['FinalScore'].replace('Eighty', 80)
```

**Figure 3.16:** Fixing the columns with inconsistencies

Figure 3.16 shows the program code to fix the columns that have inconsistencies values. We use the replace function to apply to all columns that have inconsistencies. Firstly, we apply to the Gender column where there are over three inconsistency value. Secondly, we replace the inconsistency value at the ParentEducation column which are “Bachelors” and “BSc”. Next, we fix the StudyProgram invalid value which is “-10” by using mode value. Finally, we fix the FinalScore text value which is “Eighty” to 80.

### 3.5.3 Noise and Outliers

Noise and Outliers

```
[23]: # Using NumPy library for cleaning noise and outliers
import numpy as np

# Convert FinalScore to numeric
df_cleaning['FinalScore'] = pd.to_numeric(df_cleaning['FinalScore'], errors='coerce')

# Select only the numeric columns we want to clean
cols_to_fix = ['Age', 'StudyHoursPerWeek', 'Absences', 'MidtermScore', 'FinalScore']

for col in cols_to_fix:
    Q1 = df_cleaning[col].quantile(0.25)
    Q3 = df_cleaning[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = max(0, Q1 - 1.5 * IQR)
    upper_bound = Q3 + 1.5 * IQR

    # Replace noise outliers with NaN (to fill later)
    df_cleaning.loc[(df_cleaning[col] < lower_bound) | (df_cleaning[col] > upper_bound), col] = np.nan

# Fill the new NaNs with the median
df_cleaning[cols_to_fix] = df_cleaning[cols_to_fix].fillna(df_cleaning[cols_to_fix].median())

# Change the data type
# Use .astype() now that there are no NaNs left
df_cleaning[cols_to_fix] = df_cleaning[cols_to_fix].astype('int64')

# See the statistical calculations of the fixed columns
print(df_cleaning.describe())
```

```
              Age  StudyHoursPerWeek    Absences  MidtermScore  FinalScore
count  200.000000         200.000000  200.000000    200.000000  200.000000
mean    20.360000          11.275000    3.090000     72.735000   77.575000
std      1.065871           4.870109    2.782194     15.682618   14.934173
min     19.000000           4.000000    0.000000     40.000000   45.000000
25%     19.000000           7.000000    1.000000     60.000000   69.000000
50%     20.000000          11.000000    2.000000     74.000000   80.000000
75%     21.000000          15.000000    4.000000     85.000000   89.000000
max     22.000000          22.000000   11.000000     98.000000  100.000000
```

**Figure 3.17:** Program code and output for cleaning the noise and outliers

Figure 3.17 shows the program code and output for cleaning the noise and outliers from the dataset. Firstly, we import the NumPy library for cleaning noise and outliers. Secondly, we convert the FinalScore column to numeric. Then, we create and assign the variables cols_to_fix with the numeric column names that we wanted to clean. Next, by using “for” loop, we apply the quartiles and interquartile range to fix the noise and outliers. The value that is not between lower and upper bound will be replace with NaN value. Then, we fill the NaN values with median for each column. After that, we replace the data type of each numeric column with “int64” data type. Finally, we use the “.describe()” function to display the statistical calculation of each numerical column.

```
[24]: # Check data types of cleaning dataset
      print(f'Dataset datatypes:\n\n{df_cleaning.dtypes}')

      Dataset datatypes:

      StudentID            object
      Gender               object
      Age                   int64
      StudyProgram         object
      StudyHoursPerWeek     int64
      Absences              int64
      MidtermScore          int64
      FinalScore            int64
      ParentEducation      object
      FinalGrade           object
      dtype: object

[25]: # Display the dataset cleaning
      df_cleaning
```

[25]:

| | StudentID | Gender | Age | StudyProgram | StudyHoursPerWeek | Absences | MidtermScore | FinalScore | ParentEducation | FinalGrade |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | STU001 | Female | 20 | Computer Science | 12 | 2 | 78 | 85 | Bachelor | Pass |
| **1** | STU002 | Male | 19 | Business | 8 | 5 | 65 | 70 | High School | Pass |
| **2** | STU003 | Female | 21 | Engineering | 15 | 0 | 88 | 92 | Master | Pass |
| **3** | STU004 | Male | 20 | Arts | 5 | 2 | 45 | 50 | Bachelor | Fail |
| **4** | STU005 | Female | 22 | Computer Science | 20 | 1 | 95 | 98 | PhD | Pass |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **195** | STU196 | Male | 20 | Arts | 5 | 9 | 53 | 58 | High School | Fail |
| **196** | STU197 | Female | 21 | Computer Science | 22 | 0 | 97 | 99 | PhD | Pass |
| **197** | STU198 | Male | 19 | Business | 11 | 4 | 74 | 78 | Bachelor | Pass |
| **198** | STU199 | Female | 20 | Engineering | 14 | 3 | 82 | 85 | Master | Pass |
| **199** | STU200 | Male | 21 | Arts | 6 | 5 | 56 | 61 | High School | Fail |

200 rows × 10 columns

**Figure 3.18:** Displaying dataset and its data types of each columns

Figure 3.18 shows the program code and output for displaying the datasets and its data types. In cell 24, y using the function ".dtypes" we display the datatypes for each column such as object and int64 where there are five columns with object data type and five columns with int64 data type. Next, in cell 25, we use the variable "df_cleaning" to display the dataset where there are same total rows and columns which are 200 rows and 10 columns.

```
[26]: # Verify cleaning columns
print(f"Gender: {df_cleaning['Gender'].unique()}")
print(f"ParentEducation: {df_cleaning['ParentEducation'].unique()}")
print(f"StudyProgram: {df_cleaning['StudyProgram'].unique()}")
print(f"Absences: {df_cleaning['Absences'].unique()}")
print(f"MidtermScore: {df_cleaning['MidtermScore'].unique()}")
print(f"FinalScore: {df_cleaning['FinalScore'].unique()}")

Gender: ['Female' 'Male']
ParentEducation: ['Bachelor' 'High School' 'Master' 'PhD']
StudyProgram: ['Computer Science' 'Business' 'Engineering' 'Arts']
Absences: [ 2  5  0  1  3  8  4  6 10  7 11  9]
MidtermScore: [78 65 88 45 95 70 82 55 75 68 91 40 80 60 85 50 98 72 84 74 79 67 81 42
 94 71 87 56 76 64 90 48 69 86 52 96 73 83 59 77 66 89 44 93 54 92 41 61
 51 97 57 46 47 53 58 62 43 63 49]
FinalScore: [ 85  70  92  50  98  72  78  74  95  45  84  65  89  55  99  76  88  86
  71  87  48  97  75  90  60  80  69  94  52  73  57  83  93  49  96  58
  77  46  66  56  62  51  91  59  81  53  79  47  67 100  64  61  82  68
  54  63]
```

**Figure 3.19:** Displaying the values of verified cleaning columns

Figure 3.19 shows the program code for displaying the values of verified cleaning columns. We used the ".unique()" function to display the unique values from the cleaned columns to find whether there are still missing, outliers, or noise values. As a result, all the columns have been successfully cleaned from missing, outliers, and noise values.

```
[27]: # Total Unique values from cleaning columns
print('-------------')
print(df_cleaning['Age'].value_counts())
print('-------------')
print(df_cleaning['Gender'].value_counts())
print('-------------')
print(df_cleaning['ParentEducation'].value_counts())
print('-------------')
print(df_cleaning['StudyProgram'].value_counts())
print('-------------')
print(df_cleaning['Absences'].value_counts())
print('-------------')
print(df_cleaning['FinalScore'].value_counts())
print('-------------')

-------------
Age
20    56
19    54
21    54
22    36
Name: count, dtype: int64
-------------
Gender
Female    100
Male      100
Name: count, dtype: int64
-------------
ParentEducation
Bachelor       71
High School    51
Master         48
PhD            30
Name: count, dtype: int64
-------------
StudyProgram
Arts                56
Computer Science    49
Business            48
Engineering         47
Name: count, dtype: int64
```

**Figure 3.20:** Displaying the value counts from cleaning columns (Part 1)

Figure 3.20 shows the program code and output for displaying the values counts from cleaning columns. Firstly, there are over 56 students that are 20 years old, 54 students are 19 years old, 54 students are 21 years old and 36 students are 22 years old. Secondly, there are over 100 male students and 100 female students. Next, for parent education, there 70 students with Bachelor, 51 students with High School, 48 students with Master and 30 students with PhD. Furthermore, for study program, over 56 students take Arts, 49 students take Computer Science, 48 students take Business, and 47 students take Engineering.

```
-------------
Absences
2      55
0      30
1      29
3      22
5      16
4      15
6       8
8       5
10      5
7       5
11      5
9       5
Name: count, dtype: int64
-------------
FinalScore
85      18
86      10
99       7
90       7
70       7
72       6
78       6
89       6
98       6
74       5
87       5
84       5
96       5
75       5
97       5
73       4
59       4
80       4
58       4
77       4
88       4
76       4
92       4
95       4
71       3
60       3
55       3
```

**Figure 3.21:** Displaying the value counts from cleaning columns (Part 2)

Figure 3.21 shows the value counts for Absences column and FinalScore column. For Absences column, there are over 190 students have absences 10 or below and the other 10 students have absences above 10. For FinalScore column, the mark with the highest total students is 85 marks with 18 students while the mark with the lowest total students is 54, 68, 82, 47, 64, 100, 67, 62, and 63 as shown in Figure 3.22.

```
50      3
49      3
91      3
56      2
53      2
51      2
66      2
79      2
81      2
61      2
52      2
46      2
45      2
83      2
57      2
94      2
69      2
48      2
65      2
93      2
54      1
68      1
82      1
47      1
64      1
100     1
67      1
62      1
63      1
Name: count, dtype: int64
-------------
```

**Figure 3.22:** Displaying the value counts from cleaning columns – FinalScore

Figure 3.22 shows the continuous values counts for FinalScore column.

```
-------------
MidtermScore
82      8
70      6
74      6
68      6
78      5
85      5
87      5
83      5
81      5
77      5
84      5
72      5
86      5
95      5
88      5
97      4
```

**Figure 3.23:** Displaying the value counts from cleaning columns - MidtermScore (Part 1)

```
69      4
91      4
75      4
65      4
55      4
80      4
56      4
73      4
57      3
96      3
93      3
92      3
59      3
66      3
54      3
89      3
42      3
71      3
```

**Figure 3.24:** Displaying the value counts from cleaning columns - MidtermScore (Part 2)

```
94      3
67      3
79      3
98      3
50      3
60      3
45      3
46      2
47      2
53      2
61      2
58      2
51      2
64      2
41      2
44      2
52      2
48      2
```

**Figure 3.25:** Displaying the value counts from cleaning columns - MidtermScore (Part 3)

```
90      2
76      2
40      2
62      1
43      1
63      1
49      1
Name: count, dtype: int64
-------------
```

**Figure 3.26:** Displaying the value counts from cleaning columns - MidtermScore (Part 4)

Figure 3.23, Figure 3.24, Figure 3.25 and Figure 3.26 shows the value counts for MidtermScore column. For this column, the score with the highest frequency is 82 with 8 students and the scores with the lowest frequency which are 62, 43, 63, and 49 which is only 1 student for each of them.

### 3.5.4 Normalisation of Numerical Attributes

Normalisation

```
[28]: # Import MinMaxScaler from sklearn.preprocessing library for normalisation
from sklearn.preprocessing import MinMaxScaler
# Normalise column names
cols_to_normalise = ['Age', 'StudyHoursPerWeek', 'Absences', 'MidtermScore', 'FinalScore']
# To normalise column
scaler = MinMaxScaler()
normalise_cols = [c + '_scaled' for c in cols_to_normalise]
df_normalising = df_cleaning.copy()
df_normalising[normalise_cols] = scaler.fit_transform(df_normalising[cols_to_normalise])

print(df_normalising[normalise_cols])
```

```
     Age_scaled  StudyHoursPerWeek_scaled  Absences_scaled  \
0      0.333333                  0.444444         0.181818
1      0.000000                  0.222222         0.454545
2      0.666667                  0.611111         0.000000
3      0.333333                  0.055556         0.181818
4      1.000000                  0.888889         0.090909
..          ...                       ...              ...
195    0.333333                  0.055556         0.818182
196    0.666667                  1.000000         0.000000
197    0.000000                  0.388889         0.363636
198    0.333333                  0.555556         0.272727
199    0.666667                  0.111111         0.454545

     MidtermScore_scaled  FinalScore_scaled
0               0.655172           0.727273
1               0.431034           0.454545
2               0.827586           0.854545
3               0.086207           0.090909
4               0.948276           0.963636
..                   ...                ...
195             0.224138           0.236364
196             0.982759           0.981818
197             0.586207           0.600000
198             0.724138           0.727273
199             0.275862           0.290909

[200 rows x 5 columns]
```

**Figure 3.27:** Normalising the five columns

Figure 3.27 shows normalising process for five numerical columns which are Age, StudyHoursPerWeek, Absences, MidTermScore and FinalScore. Firstly, we import the MinMaxScaler from “sklearn.preprocessing” library for normalisation purposes. Secondly, we create a list of columns names to normalise. Then, we assign create an object of MinMaxScaler() which “scaler” to be used as scaler later. Next, in “normalised_cols”, we assign the new column names follow with “_scaled” by using “for” loop. Besides that, we create a copy of cleaning dataset which is “df_normalising” so that it won’t modify the cleaning dataset and data frame and it will create the new dataset and data frame instead. Next, we apply the normalise function to the normalised columns by using “.fit_transform”. Finally, we will print out the result of the normalised columns which are Age_scaled, StudyHoursPerWeek_scaled, Absences_scaled, MidtermScore_scaled and FinalScore_scaled.

```
[29]: # Save the finaled cleaned and normalised dataset (After all cleaning and normalised)
      df_cleaning.to_csv('Edu_Data_Cleaned.csv', index=False)
      df_normalising.to_csv('Edu_Data_Normalised.csv', index=False)

[30]: # Assign and read the new cleaned dataset
      df_cleaned = pd.read_csv('Edu_Data_Cleaned.csv')
      df_cleaned
```

[30]:

| | StudentID | Gender | Age | StudyProgram | StudyHoursPerWeek | Absences | MidtermScore | FinalScore | ParentEducation | FinalGrade |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | STU001 | Female | 20 | Computer Science | 12 | 2 | 78 | 85 | Bachelor | Pass |
| **1** | STU002 | Male | 19 | Business | 8 | 5 | 65 | 70 | High School | Pass |
| **2** | STU003 | Female | 21 | Engineering | 15 | 0 | 88 | 92 | Master | Pass |
| **3** | STU004 | Male | 20 | Arts | 5 | 2 | 45 | 50 | Bachelor | Fail |
| **4** | STU005 | Female | 22 | Computer Science | 20 | 1 | 95 | 98 | PhD | Pass |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **195** | STU196 | Male | 20 | Arts | 5 | 9 | 53 | 58 | High School | Fail |
| **196** | STU197 | Female | 21 | Computer Science | 22 | 0 | 97 | 99 | PhD | Pass |
| **197** | STU198 | Male | 19 | Business | 11 | 4 | 74 | 78 | Bachelor | Pass |
| **198** | STU199 | Female | 20 | Engineering | 14 | 3 | 82 | 85 | Master | Pass |
| **199** | STU200 | Male | 21 | Arts | 6 | 5 | 56 | 61 | High School | Fail |

200 rows × 10 columns

**Figure 3.28:** Creating and saving new dataset files and display cleaned dataset

Figure 3.28 shows the program code and output for creating the new dataset files and also display the cleaned dataset. In cell 29, we save the cleaned dataset to csv file with "Index" equals to false and the name of the file is "Edu_Data_Cleaned.csv" while the normalised dataset also the same with its name of the file as "Edu_Data_Normalised.csv". In cell 30, we display the cleaned dataset by creating the variable "df_cleaned" and assign the value of its reading the csv file by using the ".read_csv()" function.

```
[31]: # Assign and read the normalised dataset
      df_normalised = pd.read_csv('Edu_Data_Normalised.csv')
      df_normalised
```

[31]:

| | StudentID | Gender | Age | StudyProgram | StudyHoursPerWeek | Absences | MidtermScore | FinalScore | ParentEducation | FinalGrade | Age_scaled | StudyHoursI |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | STU001 | Female | 20 | Computer Science | 12 | 2 | 78 | 85 | Bachelor | Pass | 0.333333 | |
| 1 | STU002 | Male | 19 | Business | 8 | 5 | 65 | 70 | High School | Pass | 0.000000 | |
| 2 | STU003 | Female | 21 | Engineering | 15 | 0 | 88 | 92 | Master | Pass | 0.666667 | |
| 3 | STU004 | Male | 20 | Arts | 5 | 2 | 45 | 50 | Bachelor | Fail | 0.333333 | |
| 4 | STU005 | Female | 22 | Computer Science | 20 | 1 | 95 | 98 | PhD | Pass | 1.000000 | |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| 195 | STU196 | Male | 20 | Arts | 5 | 9 | 53 | 58 | High School | Fail | 0.333333 | |
| 196 | STU197 | Female | 21 | Computer Science | 22 | 0 | 97 | 99 | PhD | Pass | 0.666667 | |
| 197 | STU198 | Male | 19 | Business | 11 | 4 | 74 | 78 | Bachelor | Pass | 0.000000 | |
| 198 | STU199 | Female | 20 | Engineering | 14 | 3 | 82 | 85 | Master | Pass | 0.333333 | |
| 199 | STU200 | Male | 21 | Arts | 6 | 5 | 56 | 61 | High School | Fail | 0.666667 | |

200 rows × 15 columns

**Figure 3.29:** Displaying the normalised dataset (Part 1)

Figure 3.29 shows normalised dataset. “df_normalised” was created and initiated with the value of reading csv file function “.read_csv()” with the file name of normalised data. Here we can see there are over 200 rows and 15 columns where the additional 5 columns are the normalised data columns. It shows from StudentID column until the Age_scaled column.

```
[31]: # Assign and read the normalised dataset
      df_normalised = pd.read_csv('Edu_Data_Normalised.csv')
      df_normalised
```

[31]:

| es | MidtermScore | FinalScore | ParentEducation | FinalGrade | Age_scaled | StudyHoursPerWeek_scaled | Absences_scaled | MidtermScore_scaled | FinalScore_scaled |
|---|---|---|---|---|---|---|---|---|---|
| 2 | 78 | 85 | Bachelor | Pass | 0.333333 | 0.444444 | 0.181818 | 0.655172 | 0.727273 |
| 5 | 65 | 70 | High School | Pass | 0.000000 | 0.222222 | 0.454545 | 0.431034 | 0.454545 |
| 0 | 88 | 92 | Master | Pass | 0.666667 | 0.611111 | 0.000000 | 0.827586 | 0.854545 |
| 2 | 45 | 50 | Bachelor | Fail | 0.333333 | 0.055556 | 0.181818 | 0.086207 | 0.090909 |
| 1 | 95 | 98 | PhD | Pass | 1.000000 | 0.888889 | 0.090909 | 0.948276 | 0.963636 |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 9 | 53 | 58 | High School | Fail | 0.333333 | 0.055556 | 0.818182 | 0.224138 | 0.236364 |
| 0 | 97 | 99 | PhD | Pass | 0.666667 | 1.000000 | 0.000000 | 0.982759 | 0.981818 |
| 4 | 74 | 78 | Bachelor | Pass | 0.000000 | 0.388889 | 0.363636 | 0.586207 | 0.600000 |
| 3 | 82 | 85 | Master | Pass | 0.333333 | 0.555556 | 0.272727 | 0.724138 | 0.727273 |
| 5 | 56 | 61 | High School | Fail | 0.666667 | 0.111111 | 0.454545 | 0.275862 | 0.290909 |

**Figure 3.30:** Displaying the normalised dataset (Part 2)

Figure 3.30 shows the normalised dataset, it shows from MidtermScore column until FinalScore_scaled column.

## 3.6 Improvements of Accuracy and Reliability of Data Mining Models

### *3.6.1 Handling Missing Values*

The process of preprocessing helps improve accuracy by tackling any presence of null values in the data set. For example, in the Edu_data data set, missing values for attributes like ParentEducation and StudyProgram would have been problematic and could have resulted in misleading findings or poor model outcomes. Through the process of imputation of the missing values using methods like medians and modes, the data set will be completed and consistent. It also avoids dropping records in the algorithms because of incomplete information, resulting in a better and reliable output (Koukaras & Tjortjis, 2025).

### *3.6.2 Correcting Data Inconsistencies*

Inconsistencies in data like the presence of various representations of gender ("M," "Male," "F," "Female") and score representation can lead to confusion in algorithms and negatively affect the precision of classification or clustering outcomes. Data preprocessing helps standardize all values into consistent categories, thus creating consistency within the data set. This contributes to improved accuracy since the model will be able to recognize trends without getting distracted by the inconsistent naming (Koukaras & Tjortjis, 2025).

### *3.6.3 Removing Outliers*

Outliers are data points which make the analysis unreliable and make predictions inaccurate. From the Edu_data dataset, there were outlier values in features like Age, Study Hours Per Week, and Absences. Elimination or correction of the outliers is important to ensure that the model learns from representative data, thus enhancing the accuracy of prediction. This is because using datasets with skewed distributions can cause models to predict using inaccurate data (Koukaras & Tjortjis, 2025).

### *3.6.4 Normalization of Numerical Attributes*

Through normalization, consistency is guaranteed with respect to the scaling of the features to prevent any favoritism in the algorithms towards features with high values. For instance, when a feature such as FinalScore is normalized against another feature with lower values such as StudyHoursPerWeek, there will be a balance among the features in the data set. This guarantees accuracy in the prediction process since convergence is achieved when learning takes place (Koukaras & Tjortjis, 2025).

### *3.6.5 Ensuring Fairness and Reproducibility*

Moreover, preprocessing improves the validity of models through the promotion of fairness and reproducibility. Data cleansing helps remove structural bias, whereas automated pipelines for preprocessing ensure the use of consistent processes across studies. The consistency in results improves their validity since they are replicable when using similar methods. Moreover, the problem of fairness improves since preprocessing ensures that some subgroups are not unfairly disadvantaged because of missing values, inconsistencies, and skewness (Koukaras & Tjortjis, 2025).

### *3.6.6 Overall Impact*

The purpose of data preprocessing is to convert unstructured, inaccurate, and incomplete data into structured data that increases precision and correctness. Through preprocessing, missing values can be addressed, inconsistencies corrected, outliers detected, and attribute normalization performed. The result is accurate conclusions drawn from data mining models. Data preprocessing is thus not only a preliminary process but a methodology that supports successful data mining (Koukaras & Tjortjis, 2025).

## 3.7 Insights from Data Analysis

### *3.7.1 Basic Statistical Analysis*

Insights from Data Analysis

[32]:
```
# For plotting bar graphs and scatter plots
import matplotlib.pyplot as plt
print("--- BASIC STATISTICAL ANALYSIS ---")
print(df_cleaned.describe().T)
```

```
--- BASIC STATISTICAL ANALYSIS ---
                   count    mean        std   min   25%   50%   75%    max
Age                200.0  20.360   1.065871  19.0  19.0  20.0  21.0   22.0
StudyHoursPerWeek  200.0  11.275   4.870109   4.0   7.0  11.0  15.0   22.0
Absences           200.0   3.090   2.782194   0.0   1.0   2.0   4.0   11.0
MidtermScore       200.0  72.735  15.682618  40.0  60.0  74.0  85.0   98.0
FinalScore         200.0  77.575  14.934173  45.0  69.0  80.0  89.0  100.0
```

**Figure 3.31:** Basic statistical analysis

Figure 3.31 shows the basic statistical analysis of the cleaned data frame where it do the statistical for numerical columns such as Age, StudyHoursPerWeek, Absences, MidtermScore and FinalScore. It shows the count, mean, standard deviation, minimum, 25% of the data, 50% of the data, 75% of the data and maximum of the data.

```
[33]: plt.figure(figsize=(13,7))
      plt.subplot(1,2,1)
      plt.subplots_adjust(hspace=1)
      plt.pie(df_cleaned['StudyProgram'].value_counts(), labels=df_cleaned['StudyProgram'].unique(),
             autopct='%1.2f%%')
      plt.title("Percentages of Students by Study Program")
      plt.legend()
      plt.subplot(1,2,2)
      plt.subplots_adjust(hspace=1)
      bars = plt.bar(df_cleaned['Gender'].unique(), df_cleaned['Gender'].value_counts())
      plt.bar_label(bars, padding=3)
      plt.title("Total Students by Gender")
      plt.xlabel("Gender")
      plt.ylabel("Total")
      plt.tight_layout()
      plt.grid(axis='y', linestyle='--', alpha=0.6)
      plt.savefig('PercentageOfStudentsByStudyProgram_TotalStudentsByGender.png')
      plt.show()
```

**Figure 3.32:** Program code for creating pie chart and bar chart

Figure 3.32 shows the program code for creating the pie chart and bar chart, we use the matplotlib library to plot them, using “pie()” function to create the pie chart, “bar()” function to create a bar graph and use the figure size to place them side by side as shown in Figure 3.33.

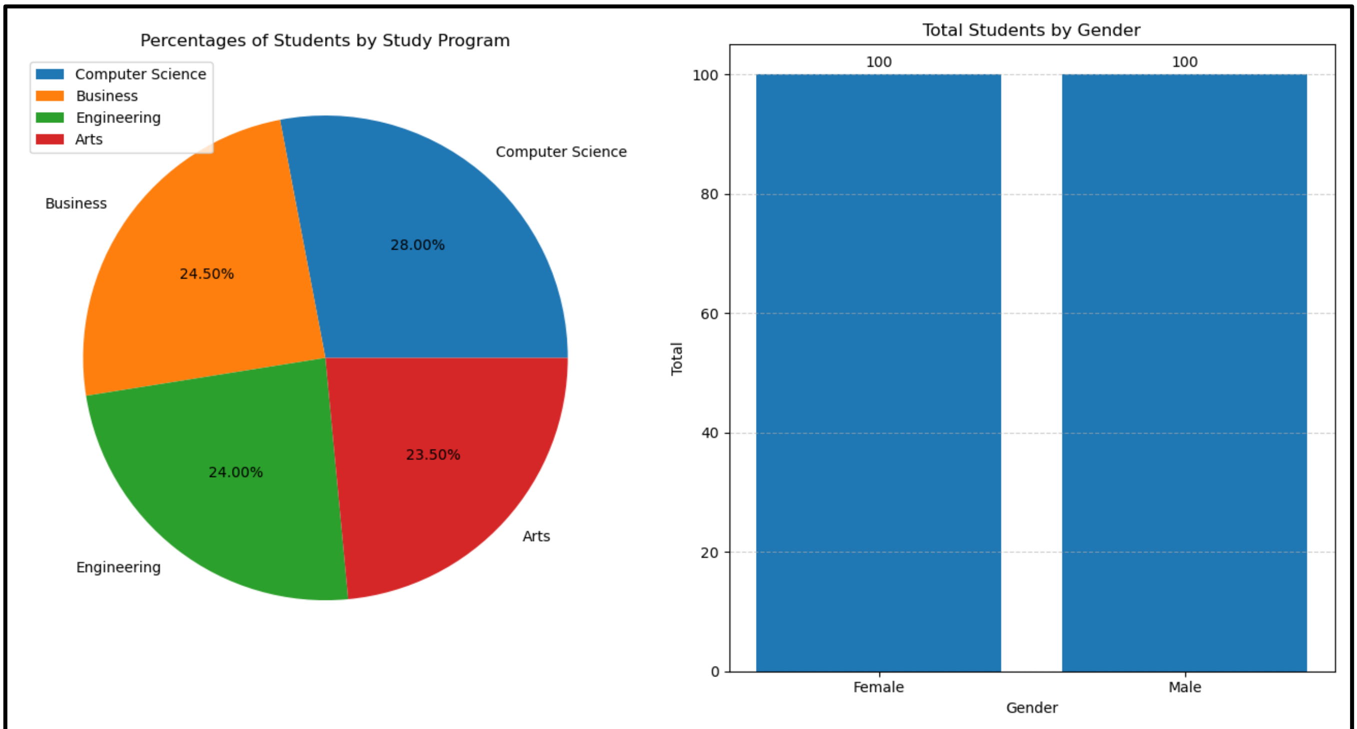



**Figure 3.33:** Percentages of Student by Study Program (left) and Total Students by Gender (right)

Figure 3.33 shows the pie chart for Percentages of Student by Study Program and bar graph for Total Students by Gender. The pie chart (left) shows of percentages of students by study program. Computer Science has the highest percentage which is 28% while the Arts has the lowest percentage which is 23.50%. The next one is the bar graph for Total Students by Gender (right). The gender Male is 100 students same as the Female which is also 100 students.

```
[34]: plt.figure(figsize=(13,7))
      plt.subplot(1,2,1)
      x1 = df_cleaned['StudyHoursPerWeek']
      y1 = df_cleaned['Absences']
      slope, intercept = np.polyfit(x1, y1, 1)
      trendline_model = np.poly1d([slope, intercept])
      plt.scatter(x1, y1, alpha=0.6, label='Data Point')
      plt.plot(x1, trendline_model(x1), color='red', linewidth=2, label='Trend Line')
      plt.title("Study Hours vs Absences")
      plt.xlabel("Study Hours Per Week")
      plt.ylabel("Absences")
      plt.legend(loc='upper right')
      plt.grid(True, linestyle='--', alpha=0.5)
      plt.subplot(1,2,2)
      y2 = df_cleaned['FinalScore']
      slope, intercept = np.polyfit(x1, y2, 1)
      trendline_model = np.poly1d([slope, intercept])
      plt.scatter(x1, y2, alpha=0.6, label='Data Point')
      plt.plot(x1, trendline_model(x1), color='red', linewidth=2, label='Trend Line')
      plt.title("Study Hours vs Final Score")
      plt.xlabel("Study Hours Per Week")
      plt.ylabel("Final Score")
      plt.legend(loc='upper left')
      plt.grid(True, linestyle='--', alpha=0.5)
      plt.tight_layout()
      plt.savefig('StudyHoursVsAbsences_StudyHoursVsFinalScore.png')
      plt.show()
```

**Figure 3.34:** Program code for creating multiple scatter plots

Figure 3.34 shows the program code for creating multiple scatter plots, we use the matplotlib library to plot them, using "scatter()" functions and use the figure size to place them side by side as shown in Figure 3.35.

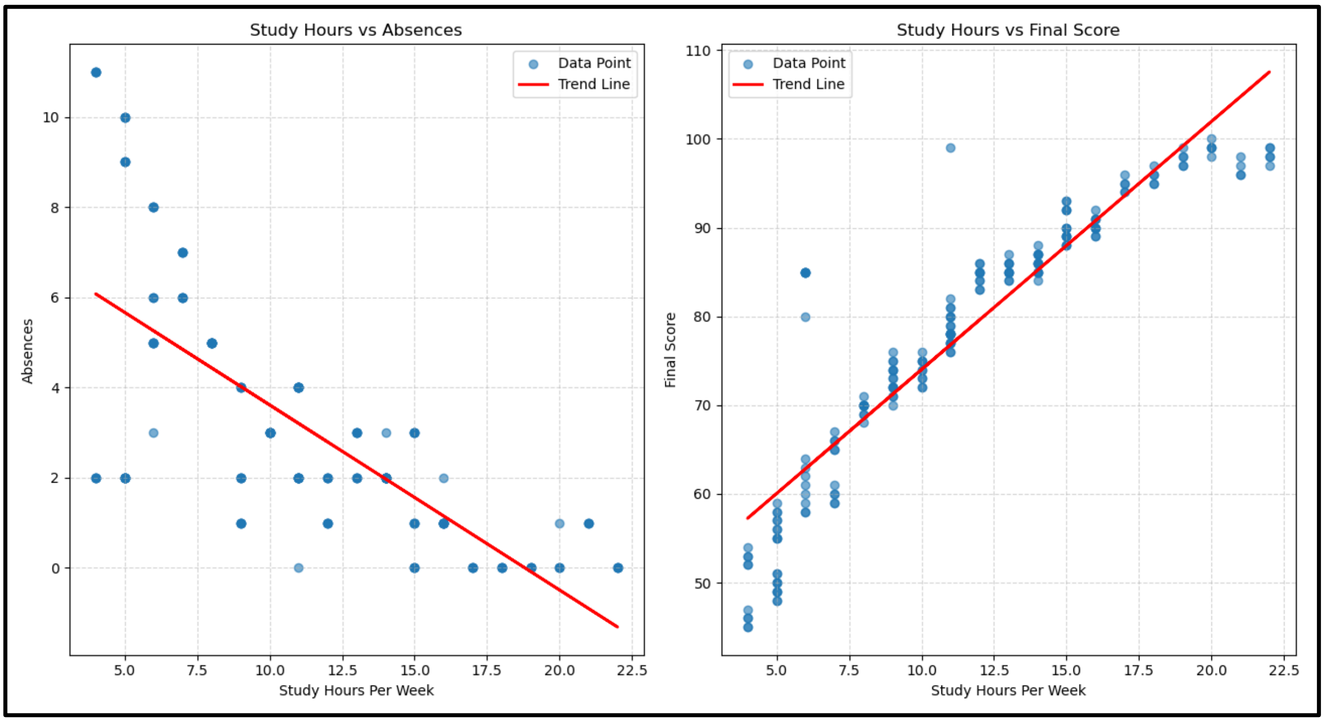



**Figure 3.35:** Study Hours vs Absences (left) and Study Hours vs Final Score (right)

Figure 3.35 shows the scatter plots for Study Hours vs Absences (left) and Study Hours vs Final Score (right). The first one which is the scatter plot for Study Hours vs Absences (left) is the negative correlation where the trend line is going from positive to negative which stated that the students who study more hours have a high chance of lowering their total absences rate. Next is the scatter plot for Study Hours vs Final Score (right) where it shows the positive correlation from lower data point to upper data point which stated that the students who study more hours gets the higher final score marks.

```
[35]: # For clustered bar charts
import seaborn as sns
plt.figure(figsize=(13,7))
plt.subplot(1,2,1)
x1 = df_cleaned['StudyHoursPerWeek']
y1 = df_cleaned['MidtermScore']
slope, intercept = np.polyfit(x1, y1, 1)
trendline_model = np.poly1d([slope, intercept])
plt.scatter(x1, y1, alpha=0.6, label='Data Point')
plt.plot(x1, trendline_model(x1), color='red', linewidth=2, label='Trend Line')
plt.title("Study Hours vs Mid Term Score")
plt.xlabel("Study Hours Per Week")
plt.ylabel("Mid Term Score")
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.5)
plt.subplot(1,2,2)
sns.countplot(data=df_cleaned, x='StudyProgram', hue='ParentEducation')
ax2 = plt.gca()
for container in ax2.containers:
    ax2.bar_label(container, padding=3)
plt.title('Comparison of Parent Education Levels by Study Program')
plt.xlabel('Study Program')
plt.ylabel('Number of Students')
plt.xticks(rotation=45)
plt.legend(title='Parent Education', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.savefig('StudyHoursVsMidTermScore_ComparisonOfParentEducationLevelsByStudyProgram.png')
plt.show()
```

**Figure 3.36:** Program code for creating scatter plot and clustered bar graphs

Figure 3.36 shows the program code for creating scatter plot and clustered bar graphs, we use the matplotlib library to plot them, using a “scatter()” function, and seaborn library to create the clustered bar graphs and use the figure size to place them side by side as shown in Figure 3.37.

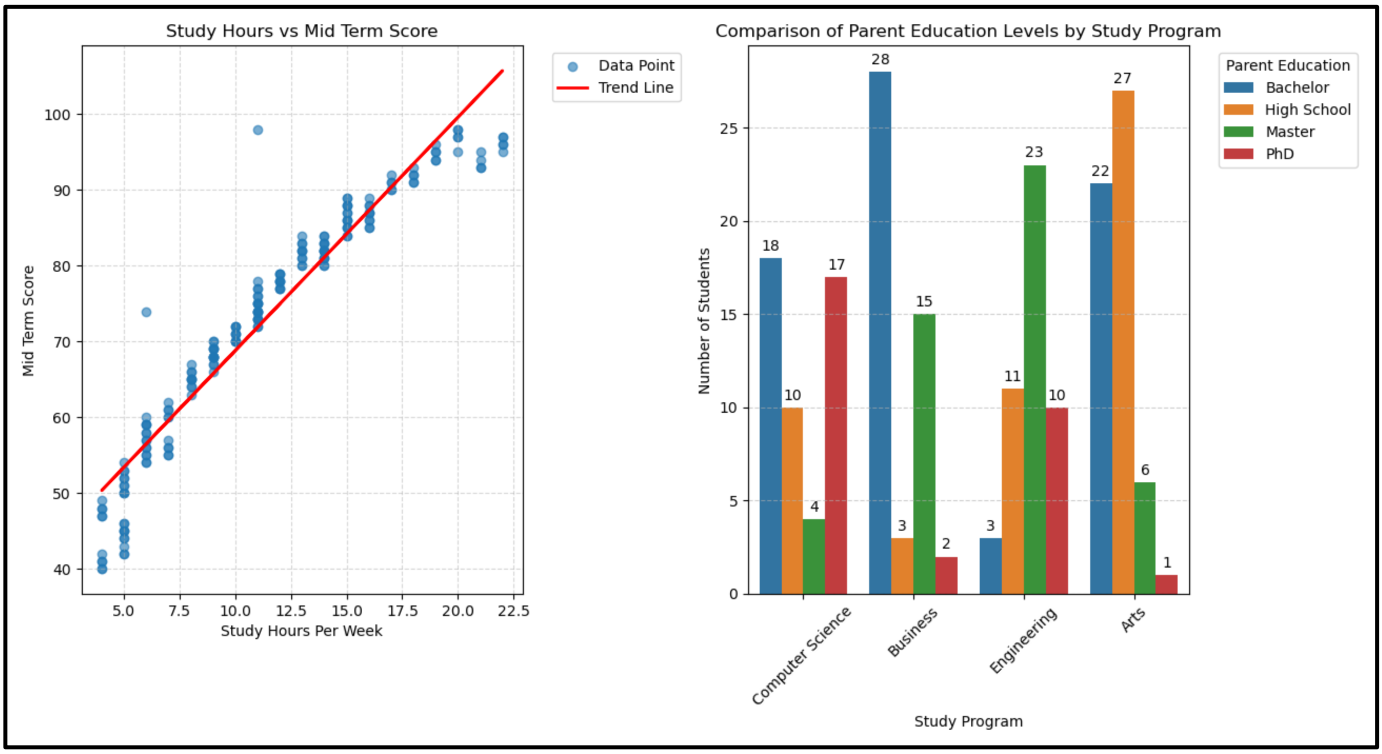
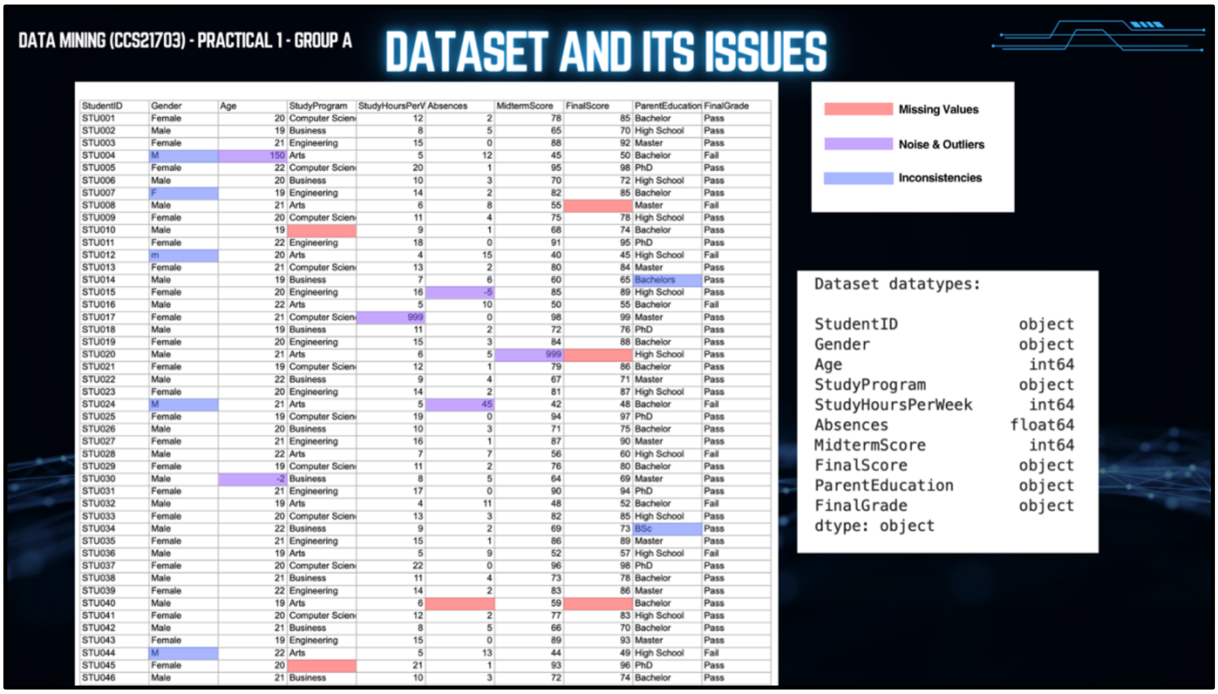


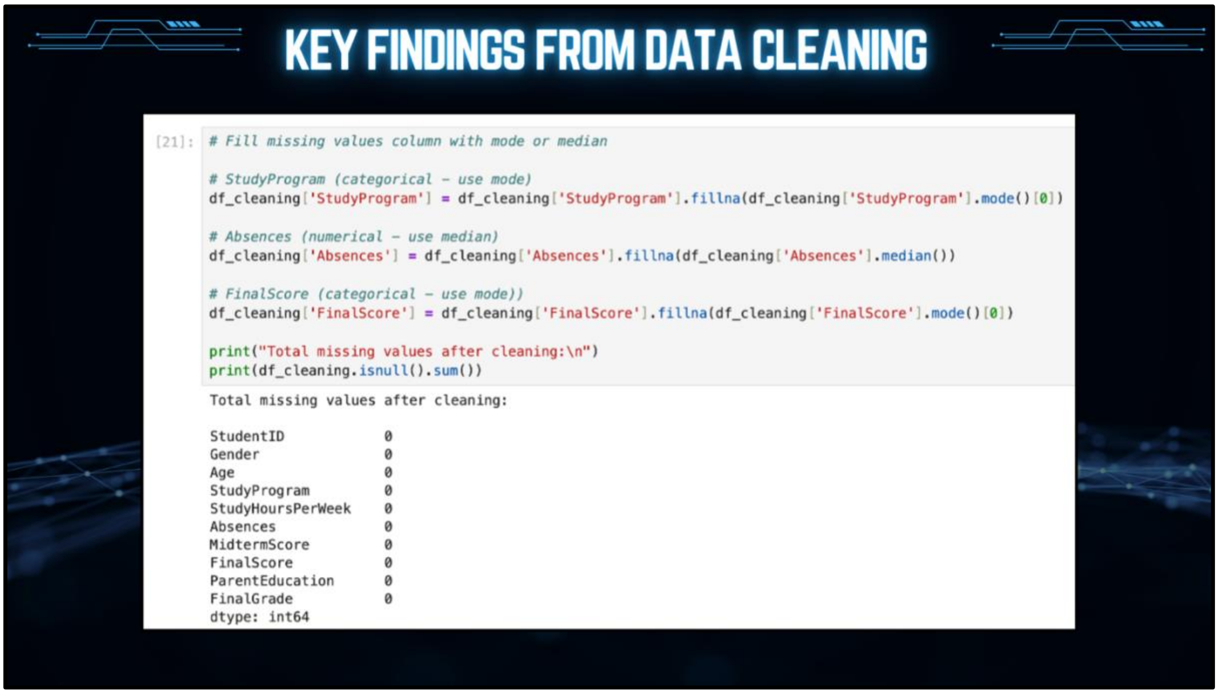

**Figure 3.37:** Study Hours vs Mid Term Score (left) and Comparison of Parent Education Levels by Study Program (right)

Figure 3.37 shows the scatter plots for Study Hours vs Mid Term Score (left) and Comparison of Parent Education Levels by Study Program (right). The first scatter plot is the Study Hours vs Mid Term Score (left) shows the positive correlation where the trend line goes from lower data point to upper data point which states that the students who study more hours can achieve higher marks for their mid-term score. The next one is the clustered bar charts for Comparison of Parent Education Levels by Study Program (right) where the Computer Science has the Bachelor highest which is 18 students for the parent education, Business program also has the Bachelor highest which is 28 students as the parent education, Engineering has the Master highest which is 23 students as its parent education, and finally Arts has the High School highest which is 27 students as its parent education.

# 4.0 PRESENTATION AND COMMUNICATION

There are over 10 presentation slides were created.



# KEY FINDINGS FROM DATA CLEANING (CONTINUE)

Inconsistencies

```
[22]: # Fix Gender
df_cleaning['Gender'] = df_cleaning['Gender'].replace({
    'M': 'Male',
    'm': 'Male',
    'F': 'Female'
})

# Fix ParentEducation
df_cleaning['ParentEducation'] = df_cleaning['ParentEducation'].replace({
    'Bachelors': 'Bachelor',
    'BSc': 'Bachelor'
})

# Fix StudyProgram invalid value
df_cleaning['StudyProgram'] = df_cleaning['StudyProgram'].replace(-10, df_cleaning['StudyProgram'].mode()[0])

# Fix FinalScore text
df_cleaning['FinalScore'] = df_cleaning['FinalScore'].replace('Eighty', 80)
```

# KEY FINDINGS FROM DATA CLEANING (CONTINUE)

Noise and Outliers

```
[23]: # Using NumPy library for cleaning noise and outliers
import numpy as np

# Convert FinalScore to numeric
df_cleaning['FinalScore'] = pd.to_numeric(df_cleaning['FinalScore'], errors='coerce')

# Select only the numeric columns we want to clean
cols_to_fix = ['Age', 'StudyHoursPerWeek', 'Absences', 'MidtermScore', 'FinalScore']

for col in cols_to_fix:
    Q1 = df_cleaning[col].quantile(0.25)
    Q3 = df_cleaning[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = max(0, Q1 - 1.5 * IQR)
    upper_bound = Q3 + 1.5 * IQR

    # Replace outliers with NaN (to fill later)
    df_cleaning.loc[(df_cleaning[col] < lower_bound) | (df_cleaning[col] > upper_bound), col] = np.nan

# Fill the new NaNs with the median
df_cleaning[cols_to_fix] = df_cleaning[cols_to_fix].fillna(df_cleaning[cols_to_fix].median())

# Change the data type
# Use .astype() now that there are no NaNs left
df_cleaning[cols_to_fix] = df_cleaning[cols_to_fix].astype('int64')

# See the statistical calculations of the fixed columns
print(df_cleaning.describe())
```

| | Age | StudyHoursPerWeek | Absences | MidtermScore | FinalScore |
|---|---|---|---|---|---|
| count | 200.000000 | 200.000000 | 200.000000 | 200.000000 | 200.000000 |
| mean | 20.360000 | 11.275000 | 3.090000 | 72.735000 | 77.575000 |
| std | 1.065871 | 4.870109 | 2.782194 | 15.682618 | 14.934173 |
| min | 19.000000 | 4.000000 | 0.000000 | 40.000000 | 45.000000 |
| 25% | 19.000000 | 7.000000 | 1.000000 | 60.000000 | 69.000000 |
| 50% | 20.000000 | 11.000000 | 2.000000 | 74.000000 | 80.000000 |
| 75% | 21.000000 | 15.000000 | 4.000000 | 85.000000 | 89.000000 |
| max | 22.000000 | 22.000000 | 11.000000 | 98.000000 | 100.000000 |

- Change FinalScore column to numeric.
- Select numeric columns to fix the noise and outliers.
- Using Quartiles to fix the noise and outliers.
- Filling the null values with median.
- Change the data type of numeric columns to integer.

# KEY FINDINGS FROM DATA CLEANING (CONTINUE)

Normalisation

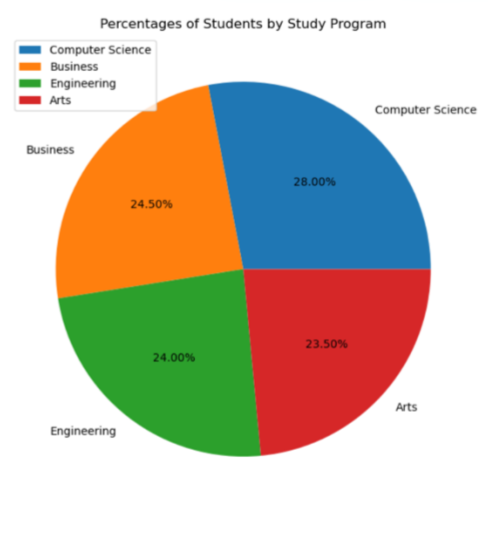
```
[28]: # Import MinMaxScaler from sklearn.preprocessing library for normalisation
from sklearn.preprocessing import MinMaxScaler
# Normalise column names
cols_to_normalise = ['Age', 'StudyHoursPerWeek', 'Absences', 'MidtermScore', 'FinalScore']
# To normalise column
scaler = MinMaxScaler()
normalise_cols = [c + '_scaled' for c in cols_to_normalise]
df_normalising = df_cleaning.copy()
df_normalising[normalise_cols] = scaler.fit_transform(df_normalising[cols_to_normalise])

print(df_normalising[normalise_cols])
```

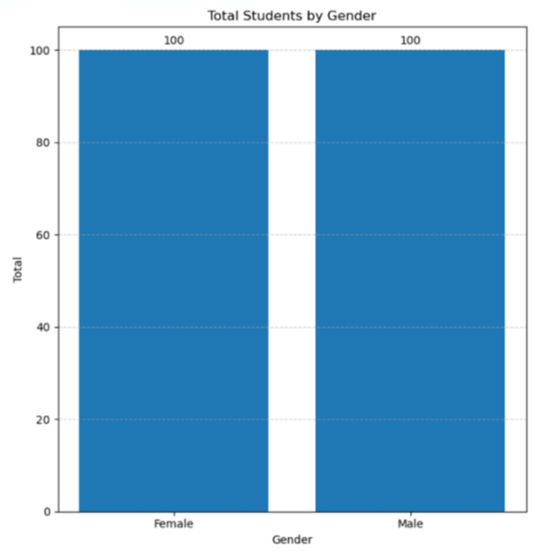
```
     Age_scaled  StudyHoursPerWeek_scaled  Absences_scaled  \
0      0.333333                  0.444444         0.181818
1      0.000000                  0.222222         0.454545
2      0.666667                  0.611111         0.000000
3      0.333333                  0.055556         0.181818
4      1.000000                  0.888889         0.090909
..          ...                       ...              ...
195    0.333333                  0.055556         0.818182
196    0.666667                  1.000000         0.000000
197    0.000000                  0.388889         0.363636
198    0.333333                  0.555556         0.272727
199    0.666667                  0.111111         0.454545

     MidtermScore_scaled  FinalScore_scaled
0               0.655172           0.727273
1               0.431034           0.454545
2               0.827586           0.854545
3               0.086207           0.090909
4               0.948276           0.963636
..                   ...                ...
195             0.224138           0.236364
196             0.982759           0.981818
197             0.586207           0.600000
198             0.724138           0.727273
199             0.275862           0.290909

[200 rows x 5 columns]
```

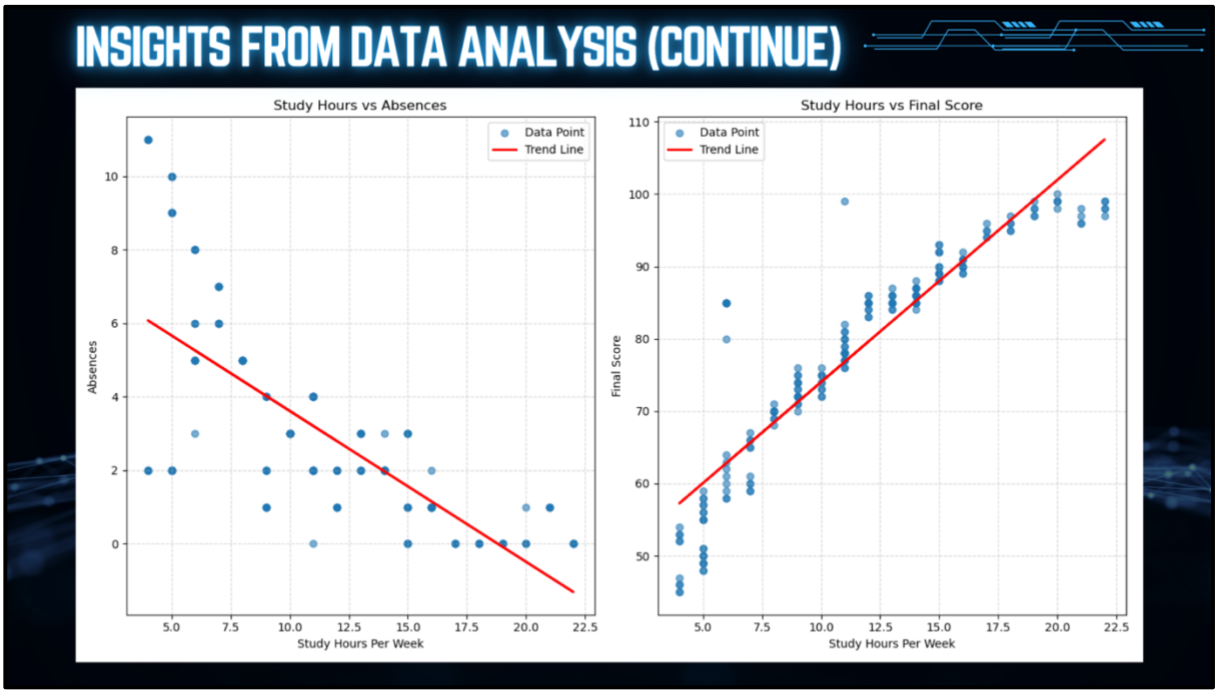
- Normalise the Age, StudyHoursPerWeek, Absences, MidtermScore and FinalScore.
- Using MinMaxScaler() object class.
- Create another copy of data frame.
- Using fit_transform to normalise the columns.
- Save into two different new files.

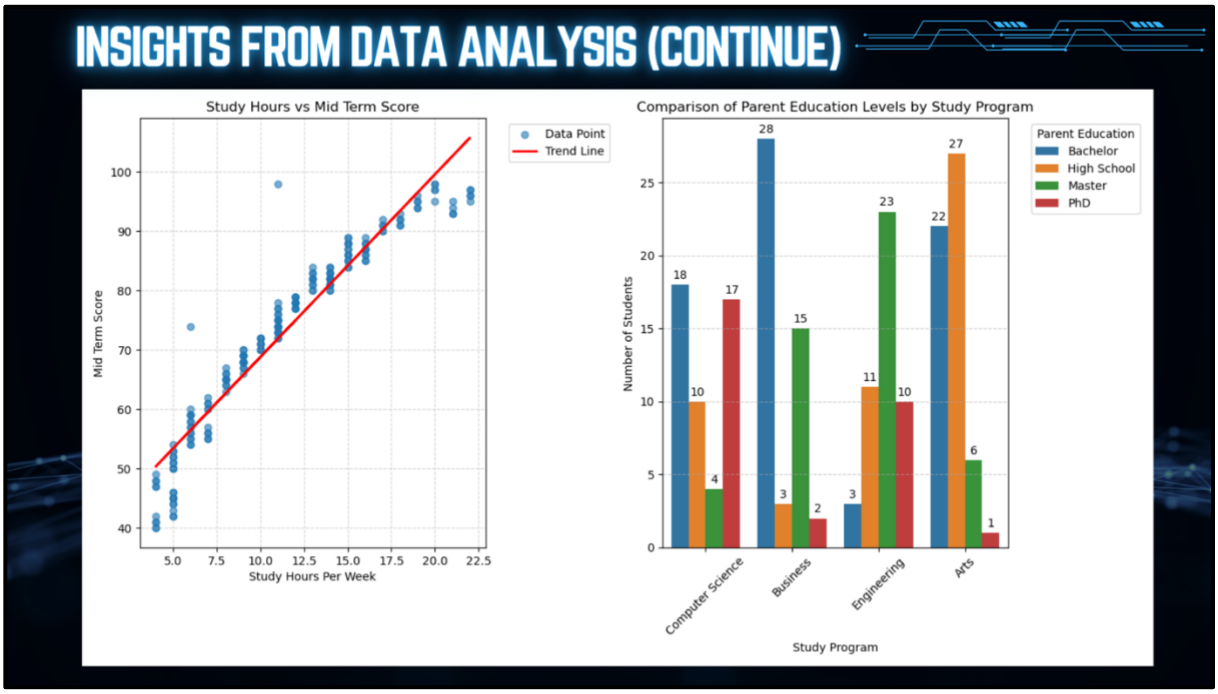
```
[29]: # Save the finaled cleaned and normalised dataset (After all cleaning and normalised)
df_cleaning.to_csv('Edu_Data_Cleaned.csv', index=False)
df_normalising.to_csv('Edu_Data_Normalised.csv', index=False)
```

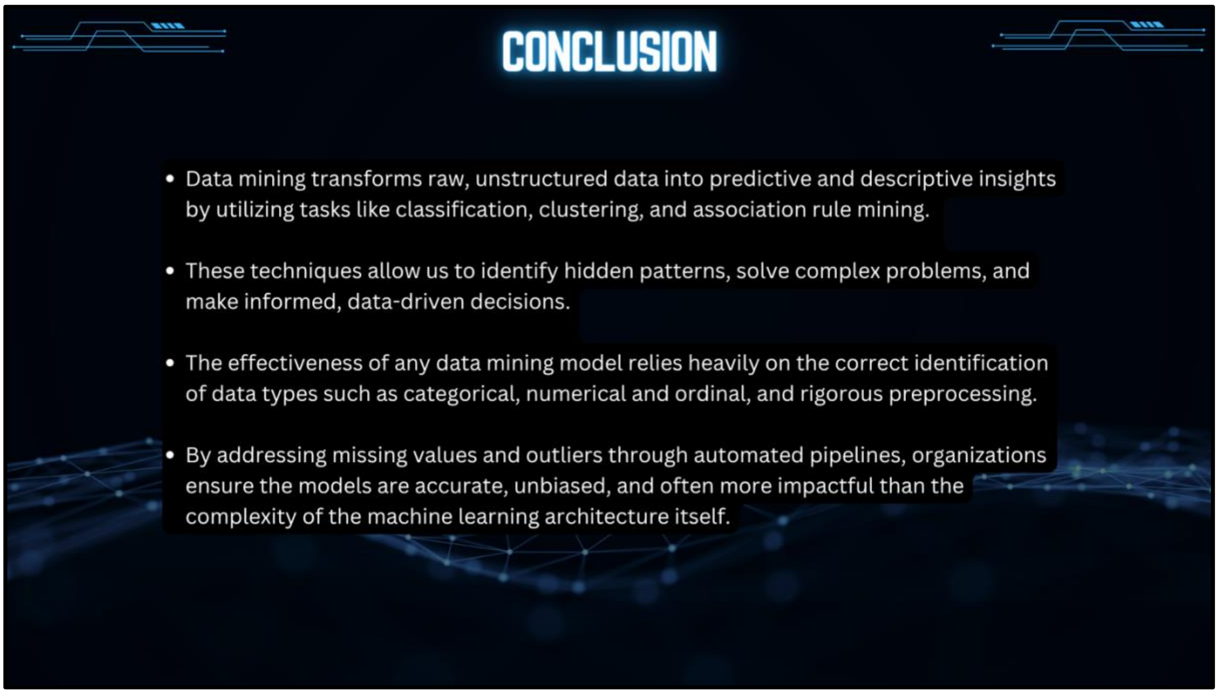
# INSIGHTS FROM DATA ANALYSIS




INSIGHTS FROM DATA ANALYSIS (CONTINUE)
Study Hours vs Absences
Data Point
Trend Line
Absences
Study Hours Per Week
Study Hours vs Final Score
Data Point
Trend Line
Final Score
Study Hours Per Week


INSIGHTS FROM DATA ANALYSIS (CONTINUE)
Study Hours vs Mid Term Score
Data Point
Trend Line
Mid Term Score
Study Hours Per Week
Comparison of Parent Education Levels by Study Program
Parent Education
Bachelor
High School
Master
PhD
Number of Students
Computer Science
Business
Engineering
Arts
Study Program


CONCLUSION
• Data mining transforms raw, unstructured data into predictive and descriptive insights by utilizing tasks like classification, clustering, and association rule mining.
• These techniques allow us to identify hidden patterns, solve complex problems, and make informed, data-driven decisions.
• The effectiveness of any data mining model relies heavily on the correct identification of data types such as categorical, numerical and ordinal, and rigorous preprocessing.
• By addressing missing values and outliers through automated pipelines, organizations ensure the models are accurate, unbiased, and often more impactful than the complexity of the machine learning architecture itself.


GROUP A
THANK YOU

# 5.0 CONCLUSION

Data mining is considered a vital part of modern-day data science, as organizations can leverage data mining techniques to convert unstructured data into intelligence. By introducing data mining by definition as an approach to using machine learning and statistical techniques in finding patterns in datasets, the report explains the evolution of data mining into an important tool of descriptive and predictive analytics that plays a crucial role in solving problems and providing organizations with the possibility to analyze past data as well as current issues.

The three key data mining tasks that include classification, clustering, and association rule mining allow one to cover the full range of analytical activities, from classification and structuring of the dataset to the identification of hidden patterns and connections between certain variables. Data mining in practice has been exemplified by several examples of its utilization in various industries, such as healthcare (in the improvement of diagnoses), retail (in running better marketing campaigns and targeting customers), education (in developing personal approaches).

It is very important to have knowledge about the different types of data such as categorical, numerical, and ordinal. This is necessary to ensure proper operations on the data, perform correct analyses, and create visualizations. Incorrect identification of data types leads to biases and errors in calculations and analysis results. Correctly recognizing the type of data will provide analysts with an opportunity to process data correctly and visualize it efficiently for informed decisions.

Also, of great significance is preprocessing that contributes to increased accuracy and reliability of the models of data mining. Preprocessing helps address problems of missing values, contradictions, and outliers in datasets and normalize data. This will lead to unbiased predictions and help ensure fairness in terms of all groups involved. Using automated pipelines in the preprocessing phase increases reproducibility of results and makes data processing more stable. In many instances, preprocessing significantly outperforms the use of complex architectures in the machine learning stage.

In conclusion, data mining uses statistical analysis, machine learning, and artificial intelligence together in an interrelated fashion to facilitate the process of making smart and well-thought-out decisions. In addition, by combining the processes of data pre-processing and knowledge about the types of data, data mining can become a very efficient, innovative, and sustainable process.

# 6.0 REFERENCES


Ashaduzzaman, M., …, Pritom, A., I. (2018, December). *AN ANALYSIS OF STUDENTS' ACADEMIC RECORD USING DATA MINING TECHNIQUES AND IDENTIFICATION OF KEY FACTORS TO AID STUDENTS' PERFORMANCE*. GUB Journal of Science and Engineering, 5(1). https://doi.org/10.13140/RG.2.2.29196.67204s

Cunha, J. L. (2024, December 14th). *5 Types of Data Visualization (With Examples)*. builtin. https://builtin.com/articles/types-of-data-visualization

Gayawan, E., Awe, O. O., Oseni, B.M., Uzochukwu, I. C., Adekunle, A., Samuel, G., Eisen, D. P., Adegboye, O.A. (2020, August 26th). *The spatio-temporal epidemic dynamics of COVID-19 outbreak in Africa*. Epidemiology and Infection 148, e212, 1–11. https://doi.org/10.1017/S0950268820001983

GeeksForGeeks. (2026, February 10th). *Cluster Analysis*. https://www.geeksforgeeks.org/data-analysis/cluster-analysis/

GeeksForGeeks. (2026, January 6th). *Classification in Data Mining*. https://www.geeksforgeeks.org/machine-learning/classification-in-data-mining/

Habib, J. M. (2024, September 18th). *Understanding Types of Variables in Data Science: Numerical and Categorical (Nominal, Ordinal, Continuous, and Discrete)*. Medium. https://medium.com/@jaberi.mohamedhabib/understanding-types-of-variables-in-data-science-numerical-and-categorical-nominal-ordinal-02d3bb60fd55

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann. https://ia800702.us.archive.org/7/items/datamining_201811/DS-book%20u5.pdf

Hashemi-Pour, C., Lutkevich, B. (2024, April 30th). *association rules*. TechTarget. https://www.techtarget.com/searchbusinessanalytics/definition/association-rules-in-data-mining

Holdsworth, J. (n.d.). *What is data mining?*. IBM. https://www.ibm.com/think/topics/data-mining

IBM. (n.d.). *What is data visualization?*. https://www.ibm.com/think/topics/data-visualization

Kholod, M., Mokrenko, N. (2024). *Market Basket Analysis Using Rule-Based Algorithms and Data Mining Technique*. ResearchGate. https://doi.org/10.13140/RG.2.2.25501.01769

Kolling, M. L., Furstenau, L. B., Sott, M. K., Rabaioli, B., Ulmi, P. H., Bragazzi, N. L., & Tedesco, L. P. C. (2021). *Data mining in healthcare: Applying strategic intelligence techniques to depict 25 years of research development*. Frontiers in Public Health, 9, 800265. https://www.mdpi.com/1660-4601/18/6/3099

Koukaras, P., & Tjortjis, C. (2025). *Data preprocessing and feature engineering for data mining: Techniques, tools, and best practices*. AI, 6(10), 257. https://doi.org/10.3390/ai6100257

Li, C. (2019, June 20). *Preprocessing methods and pipelines of data mining: An overview*. arXiv. https://arxiv.org/abs/1906.08510

Mansour, M. M., …, Elrazik, E. M. A. (2021, April 10th). *Modeling the COVID-19 Pandemic Dynamics in Egypt and Saudi Arabia*. Mathematics 2021, 9, 827. https://doi.org/10.3390/math9080827

Ramageri, B. M., & Desai, B. L. (2013). *Role of data mining in retail sector*. International Journal on Computer Science and Engineering (IJCSE), 5(1), 47–50. https://www.researchgate.net/publication/354653024_ROLE_OF_DATA_MINING_IN_RETAIL_SECTOR

Ramakrishnan, S. (2023, July–August). *The importance of data mining & predictive analysis*. International Journal of Engineering Technology and Management Sciences, 7(4), 593–598. https://www.researchgate.net/publication/373730722_The_Importance_of_Data_Mining_Predictive_Analysis

Rumsey, D. J. (2021, July 8th). *TYPES OF STATISTICAL DATA: NUMERICAL, CATEGORIAL, AND ORDINAL*. Statistics For Dummies. dummies. https://www.dummies.com/article/academics-the-arts/math/statistics/types-of-statistical-data-numerical-categorical-and-ordinal-169735/

Sundermann, A. J., …, Harrison, L. H. (2019, March). *Automated data mining of the electronic health record for investigation of healthcare-associated outbreaks*. Infect Control Hosp Epidemiol, 40(3), 314–319. https://pmc.ncbi.nlm.nih.gov/articles/PMC8189294/

Voroshchuk, D. (2021, January 13th). *Importance of understanding data types (Part I – Categorical/ Qualitative Data Types)*. The Data School. https://www.thedataschool.com.au/blogs/importance-of-understanding-data-types-categorical-qualitative-datatypes/#:~:text=When%20you%20start%20working%20with,the%20correct%20conclusions%20and%20insights.

Wicaksono, A. (2025, July 11th). *Why data types and data structures are important in Data Analysis?*. Medium. https://medium.com/@ariefwcks303/why-data-types-and-data-structures-are-important-in-data-analysis-bd40222d4023

Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). *Data mining: Practical machine learning tools and techniques* (4th ed.). Morgan Kaufmann. https://tjzhifei.github.io/links/DM3.pdf

Zhang, Y., Yun, Y., An, R., Cui, J., Dai, H., & Shang, X. (2021). *A review on student performance prediction using educational data mining techniques*. Computational Intelligence and Neuroscience, 2021, Article 8705529. https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2021.698490/full

## RUBRIC for Practical 1

## CLO2 – Assemble data mining solutions using software packages, including data analysis, modelling and validation. (P4, PLO3)

| COMPONENTS of CRITERIA/ WEIGHTAGE (%) | CRITERIA/ WEIGHTAGE (%) | LEVELS of ACHIEVEMENTS/ DESCRIPTORS | | | | | TOTAL SCORE 100 marks | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | **Excellent (4)** | **Good (3)** | **Average (2)** | **Needs Improvement (1)** | **Not Attempted (0)** | **1** | **2** | **3** | **4** |
| **Content (40%)** | **Foundational Concepts & Applications (15%)** | Defines data mining and its significance while providing a specific real-world application in healthcare, retail, or education. Correctly describes classification, clustering, and association rule mining tasks. | Defines data mining and its significance but the real-world application provided is general rather than industry-specific. Describes two out of three major data mining tasks correctly. | Defines data mining without explaining its industry significance and provides an application without a clear domain link. Describes one major data mining task correctly. | Provides a vague definition of data mining and lists tasks without any descriptions or real-world applications. | No definitions, tasks, or applications are provided in the report. | | | | |
| | **Data Understanding & Visualization (15%)** | Differentiates between categorical, numerical, and ordinal types and explains the necessity of this understanding for mining tasks. Suggests two visualization types and explains their specific purpose for distribution analysis. | Differentiates between two data types and explains the importance of understanding data. Suggests two visualization types but only explains the purpose for one. | Lists data types without explaining the differences or their importance to data mining. Suggests one visualization type with a basic explanation. | Mentions data types incorrectly and lists a visualization type without explaining its purpose or relationship to the data. | No discussion of data types or visualizations is included. | | | | |
| | **Preprocessing Logic & Improvement (10%)** | Identifies three raw data issues and discusses how preprocessing | Identifies two raw data issues and explains the importance of | Identifies one raw data issue and provides a brief statement on the | Lists preprocessing as a step but does not identify | Preprocessing importance and data | | | | |

| COMPONENTS of CRITERIA/ WEIGHTAGE (%) | CRITERIA/ WEIGHTAGE (%) | LEVELS of ACHIEVEMENTS/ DESCRIPTORS | | | | | TOTAL SCORE 100 marks | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | **Excellent (4)** | **Good (3)** | **Average (2)** | **Needs Improvement (1)** | **Not Attempted (0)** | **1** | **2** | **3** | **4** |
| | | specifically improves model accuracy and reliability. Explains the importance of preprocessing in the context of project success. | preprocessing generally. Mentions that preprocessing improves model results without specifying accuracy or reliability. | importance of preprocessing. | specific data issues or explain its impact on model performance. | issues are not discussed. | | | | |
| **Process (30%)** | **Data Cleaning Procedures (10%)** | Provides step-by-step methods for handling missing values and outliers based on the identified data issues. Documentation reflects the specific actions taken within the software environment. | Outlines methods for handling missing values and outliers but lacks one procedural step for full implementation. Documentation describes the general cleaning process. | Describes a method for handling either missing values or outliers but omits the other. Documentation is brief and lacks procedural detail. | Mentions that cleaning was performed but does not provide the specific steps or methods used for missing values or outliers. | No cleaning procedures or steps are documented. | | | | |
| | **Attribute Normalization (10%)** | Details the specific step-by-step method for the normalization of numerical attributes within the software package. The procedure aligns with the requirements of the data distribution. | Outlines the normalization process for numerical attributes but omits the specific software steps or parameters used. | Mentions normalization was applied to the attributes but does not describe the method or the steps taken. | Identifies the need for normalization but does not provide any description of a method or implementation. | Normalization processes are not mentioned or attempted. | | | | |
| | **Software Integration (10%)** | Assembles the data mining solution by integrating analysis, modeling, and validation steps into a continuous software workflow. Includes screenshots as | Integrates the data mining stages within the software but shows a break in the workflow between modeling and validation. Includes screenshots of the final output only. | Uses the software for data analysis and modeling as separate tasks without integrating them into a single solution. | Uses the software for a single task only, such as visualization, without attempting a full modeling solution. | No evidence of software use or solution assembly is provided. | | | | |

| COMPONENTS of CRITERIA/ WEIGHTAGE (%) | CRITERIA/ WEIGHTAGE (%) | LEVELS of ACHIEVEMENTS/ DESCRIPTORS | | | | | TOTAL SCORE 100 marks | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | **Excellent (4)** | **Good (3)** | **Average (2)** | **Needs Improvement (1)** | **Not Attempted (0)** | **1** | **2** | **3** | **4** |
| | | evidence of the software output. | | Screenshots are included but are unlabelled. | Screenshots are missing. | | | | | |
| **Product (30%)** | **Professional Reporting & Structure (15%)** | Includes a title page, table of contents, clear headings, and follows all formatting rules (Arial 11, 1.5 spacing, Justified). The report is organized into the four required sections. | Includes a title page and headings but the table of contents is missing or inaccurate. Follows two out of three formatting rules for font and spacing. | Includes headings but lacks a title page and table of contents. Font and spacing do not follow the specified guidelines. | Present as a single block of text without headings, title page, or adherence to formatting guidelines. | No report was submitted or the format is entirely incorrect. | | | | |
| | **Information Sharing & Reporting Quality (15%)** | Conveys technical concepts using precise, academic language and correct APA citations for all external references. Demonstrates zero errors in grammar or spelling and includes clear, labeled software output evidence. | Uses clear language with minor grammar errors that do not impact the technical meaning. Includes APA citations but contains two or fewer formatting errors in the reference list. | Uses repetitive language and provides references that deviate from the APA format. Grammatical errors are present and software evidence lacks descriptive labeling. | Relies on informal language and provides a list of raw URLs instead of formal citations. Significant spelling and grammar errors hinder the reader's understanding of the findings | No evidence of information sharing, references, or software outputs is submitted. | | | | |
| OVERALL SCORE | | | | | | | | | | |